

O QUE É?

Programação Orientada a Objetos



Aula P_11

Cheat sheet – Aula 1

SDLC (ciclo de desenvolvimento de projectos de SW):

Planeamento → Análise → Design
→ Implementação → Testes → Manutenção

Java – ideias-chave:

Tudo está dentro de uma **classe**

Classe = ideia

Objecto = coisa concreta

Java é **fortemente tipado**

VS Code

- Create Java Project
- Cria automaticamente src, lib
- Compila e executa com **Run** ►
- O VS Code gere o projecto, o **Java executa**

Ferramentas:

JDK 21 LTS → compilar e executar Java

VS Code → editor

Java Extension Pack → VS Code como IDE Java

Compilar e Executar:

```
javac Hello.java    → compila (.class)
java Hello          → executa (JVM)
```

ProjetoJava/

```
|— src/    → código fonte (.java)
|— bin/    → ficheiros compilados (.class)
|— lib/    → bibliotecas externas
```

Cheat sheet – Aula2

Cabeçalho obrigatório :

(estrutura mínima) package aula02; organiza o código por “pastas” e evita conflitos import ...;

diz ao compilador onde está a classe (ex.: Scanner) Tudo começa em: public static void main(String[] args)
O nome do ficheiro = nome da classe pública

Input/Output:

Ler do teclado:

Scanner sc = new Scanner(System.in);
Inteiros: nextInt() | Reais: nextDouble()
Linha inteira: nextLine()

Output simples: System.out.print / println
Output formatado: System.out.printf("... %.2f%n", valor);
%n = nova linha

Ciclos (quando usar qual)

- while (condição) testa antes: pode nem entrar { ... }
- while (condição); testa depois: corre pelo menos 1 vez
- Padrão típico: pedir input até ser válido (ex.: “tem de ser positivo”)

Cálculo com Math:

Distância: Math.hypot(dx, dy) (equivalente a $\sqrt{dx^2 + dy^2}$)

Pitágoras: Math.hypot(A, B)

Trigonometria em Java trabalha em radianos: Math.acos(...) devolve radianos
converter: Math.toDegrees(rad)

Listas (ArrayList) e o “modelo em 2 fases”:

ArrayList<Double> valores = new ArrayList<>();
add() guarda valores;
size() dá total

Modelo mental: recolher (guardar na lista) processar (percorrer com for-each para min/max/soma/média)

Strings e métodos estáticos (mini-caixa de ferramentas)

String em Java já existe “de fábrica” (java.lang)
Operações comuns: length(), toLowerCase(), toUpperCase(), trim(), charAt(i), substring(a,b), contains(...), replace(...)
Métodos static: pertencem à classe, não a objectos úteis para tarefas: contar dígitos, espaços, palíndromo, reduzir espaços

Cheat sheet – Aula 3

Interacção com o Utilizador (Classe Scanner)

O ponto de partida para qualquer programa que precise de dados.

Declaração: `import java.util.Scanner;`

Criação: `Scanner sc = new Scanner(System.in);`

Leitura de Linha: `String s = sc.nextLine();`

Leitura de Números: `int i = sc.nextInt();` ou `double d = sc.nextDouble();`

Propriedades e Posições (O índice 0)

É crucial que os alunos recordem que em Java a contagem começa sempre em **zero**.

Comprimento: `s.length()` (devolve o total de caracteres).

Primeiro Carácter: `s.charAt(0)`

Último Carácter: `s.charAt(s.length() - 1)` – *Esta é a fórmula que evita o erro de índice fora de limites.*

Subtracção de Partes (Método substring)

Fundamental para extrair prefixos ou fragmentos.

Sintaxe: `s.substring(inicio, fim)`

Regra de Ouro: O carácter na posição fim **não** é incluído.

Exemplo: `s.substring(0, 3)` extrai os caracteres nas posições 0, 1 e 2.

Para evitar erros de comparação devido a espaços ou letras maiúsculas.

s.trim(): Remove espaços inúteis no início e fim.

s.toLowerCase() / s.toUpperCase(): Normaliza a caixa do texto.

s.replace('a', 'b'): Substitui caracteres ou palavras.

Verificações Lógicas (Booleanos)

Útil para as estruturas de decisão (if).

s.isEmpty(): Atalho para `s.length() == 0`.

s.contains("texto"): Verifica se o padrão existe na String.

s.equals("comparacao"): **Nunca** usar `==` para comparar o conteúdo de Strings.

O Operador Ternário (Decisão Rápida)

Conforme demonstrado na Aula 3 para decidir o output numa única linha.

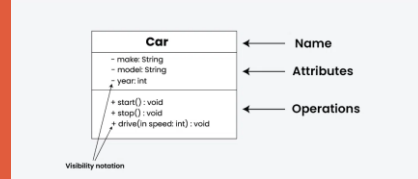
Estrutura: (condição) ? valor_se_verdade : valor_se_falso;

Exemplo: `String msg = (s.length() >= 3) ? s.substring(0,3) : s;`

Classes e Objectos

Classe = modelo

Objecto = instância concreta criada a partir da classe.



Métodos modificadores

set() permite **alterar** valores com validação.

Atributos privados

Uso de **private** para proteger os dados do objecto (encapsulamento).

Métodos especiais

toString() → representação textual
equals() → comparar objectos.

Métodos acessores

get() permite **ler** valores de atributos privados.

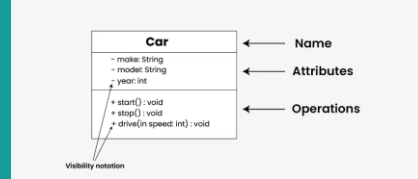
Métodos de comportamento

Métodos como area() e perimetro() implementam **o comportamento das classes**.

Classes e Objects

Classe = modelo

Objecto = instância concreta criada a partir da classe.



Métodos Construtores inicializam objectos

Responsáveis por: atribuir valores iniciais
validar argumentos, garantir consistência interna

Objectos representam entidades reais

Um objecto modela algo do mundo: calendário, data, evento, aluno
Programar em POO é modelar o mundo com classes.

Objectos podem conter estruturas internas

Exemplos:

Arrays, matrizes, listas

Permitem representar: dias, eventos, registos

Encapsulamento protege os dados

Os atributos:

são privados, não são acedidos directamente, usam métodos de consulta
Evita erros e mantém controlo sobre o estado do objecto.

Classes colaboram entre si

Classe Calendário usa Classe Data

POO não são classes isoladas. São **classes que cooperam. Agregação** (“tem um”)
Uma classe contém outra, mas ambas existem ndependentemente

Cheat sheet – Aula6

Herança (“é um”)

Permite reutilizar código entre classes relacionadas

Exemplo:

Aluno é uma Pessoa

Professor é uma Pessoa

Bolseiro é um Aluno

Agregação (“tem um”)

Uma classe contém outra, mas ambas existem independentemente

Exemplo: Pessoa tem DateYMDContacto
tem Pessoa

Não usa extends

super

Serve para chamar elementos da superclasse. Usado para:

super(...)

super.metodo()

primeira linha do construtor da subclasse

Comparable

Define a **ordem natural dos objectos**

Permite: Collections.sort(lista)

Implementação:

implements Comparable<Pessoa>
compareTo(...)

Classes especializadas

Construídas por herança incremental

Pessoa



Cada nível acrescenta comportamento

Modelação a partir da narrativa

Processo mental essencial da POO

Narrativa → Classes

Exemplo: Pessoa

Contacto

GestaoContactos

Cheat sheet – Aula 7

Classe abstracta

Classe abstracta: não pode ser instanciada
define atributos e serviços comuns
pode conter métodos abstractos
funciona como **superclasse lógica**

Exemplo:

```
abstract class Forma
```

Método abstracto = contrato

Método abstracto:
declarado sem implementação
obriga subclasses a implementar
define comportamento obrigatório

Herança aplicada às figuras

Subclasses especializam a superclasse
Cada subclasse:
reutiliza atributos comuns
implementa `area()` e `perimetro()`

Polimorfismo (ideia central)

Variável do tipo superclasse *Forma f*

Pode referenciar: *new Circulo()*
new Rectangulo()
new Triangulo()

Mesmo método → comportamento diferente

Igualdade entre objectos (override)

`equals()` redefine igualdade lógica:

Duas figuras são iguais se:

- *mesmo tipo*
- *mesmos atributos*
- *mesma cor*

Não usar `==` para comparar objectos

Abstração explícita nas datas

- *Date = conceito abstracto* `DateYMD` e `DateND` = *implementações*

programa passa a depender da abstração
e não da representação

Cheat sheet – Aula 8

Classe abstracta (revisão)

O que é: Classe que não pode ser instanciada, serve como superclasse lógica.

No exercício Berlim:

```
public abstract class Veiculo implements  
IKmPercorridos
```

Interface como contrato

O que é: Define métodos que as classes são obrigadas a implementar.

No exercício Berlim:

```
public interface  
IKmPercorridos com trajeto(), ultimoTrajeto(  
) , distanciaTotal()
```

Quem implementa: Veiculo (todas as viaturas) e Atleta (pessoa que treina)

Classe pode implementar várias interfaces

No exercício Berlim (alínea b):

```
AutomovelLigeiroEletrico extends  
AutomovelLigeiro implements IGestaoBateria
```

Uma classe extends apenas uma superclasse

No exercício Berlim:

```
AutomovelLigeiro extends Veiculo
```

```
Taxi extends AutomovelLigeiro (herança em  
cadeia)
```

```
AutomovelLigeiroEletrico extends
```

```
AutomovelLigeiro implements  
IGestaoBateria
```

Polimorfismo com interfaces

O que é: Variável do tipo interface pode referenciar qualquer classe que a implemente.

No exercício Berlim (alínea b):

```
IGestaoBateria[] dispositivos = { tel, e1 };  
(tel é Telemovel, e1 é
```

```
AutomovelLigeiroEletrico – ambos  
implementam IGestaoBateria)
```

TURMA P3

TEMA GERAL

Sistema de gestão de uma cervejaria

ENTIDADES PRINCIPAIS



Cerveja



Francesinha



Acompanhamento



Cervejaria

CONCEITOS EXPLORADOS



- Herança
- Encapsulamento
- Gestão de filas/listas
- Compatibilidade lógica
- Controlo de vendas
- Lucro total
- Regras de negócio
- Construtores
- Identificadores sequenciais
- Polimorfismo

REGRAS IMPORTANTES



- Francesinha vegetariana: tipo da carne tem de ser vegetariana
- Não permite itens duplicados (mesmo id)
- Só permite vender Acompanhamento se o último item vendido foi uma Francesinha
- Todos os itens são pagos na entrega
- Lucro = preço_venda – preço_base



FOCO PEDAGÓGICO

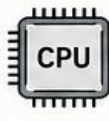
Modelação de produtos e regras comerciais simples.
Introdução clara à orientação a objetos e às regras de negócio.

TURMA P7

TEMA GERAL

Sistema de gestão de uma loja de informática

ENTIDADES PRINCIPAIS



Processador (CPU)



PlacaMae (Motherboard)



Memoria (RAM)



LojaInformatica

CONCEITOS EXPLORADOS



- Herança técnica
- Encapsulamento
- Construtores
- Compatibilidade hardware
- Gestão de stock
- Regras de compatibilidade
- Vendas condicionadas
- Polimorfismo
- Lucro total
- Impressão filtrada

REGRAS IMPORTANTES



- Se a PlacaMae for AM5, o tipo de memória tem de ser DDR5
- Não permite componentes duplicados (mesmo id)
- Só permite vender Memoria se o último componente vendido foi uma PlacaMae compatível (tipo de RAM e portátil ou não)
- Lucro = preço_venda – preço_base



FOCO PEDAGÓGICO

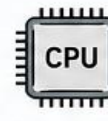
Modelação de componentes compatíveis e dependências entre objetos.
Introdução a validações entre entidades e gestão de estado.

TURMA P9 / P10

TEMA GERAL

Sistema de loja de informática (versão mais avançada)

ENTIDADES PRINCIPAIS



Processador (CPU)



PlacaMae (Motherboard)



Memoria (RAM)



LojaInformatica

CONCEITOS EXPLORADOS



- Consolidação de POO
- Herança
- Encapsulamento
- Polimorfismo
- Gestão dinâmica de objetos
- Estado interno do sistema
- Regras de negócio mais complexas
- Compatibilidades cruzadas
- Lucro total

REGRAS IMPORTANTES



- AM5 exige Memoria DDR5
- Não permite componentes duplicados (mesmo id)
- Só permite vender Memoria se o último componente vendido foi uma PlacaMae compatível
- Controla o estado do último componente vendido
- Lucro = preço_venda – preço_base



FOCO PEDAGÓGICO

Aplicação integrada de orientação a objetos com regras de negócio mais exigentes.
Maior complexidade no controlo de estado e nas validações entre objetos.

PARTE 1 (TODAS AS TURMAS)



- Leitura de dados do teclado
- Leitura de dados de ficheiro (inputs.txt)
- Parsing de:

- ✓ inteiro
- ✓ lista de inteiros
- ✓ string
- ✓ lista/string de strings

- Impressão formatada
- Utilização de Scanner
- Substituição de fonte de dados

EVOLUÇÃO PEDAGÓGICA



Contexto intuitivo (comida e bebida)



Contexto técnico hardware e compatibilidades simples



Mesmas ideias, maior exigência lógica e estrutural

SUMÁRIO DAS PRINCIPAIS DIFICULDADES DOS ALUNOS



Os alunos escrevem Java, mas muitos ainda não conseguem pensar orientado a objectos.
Entram em “modo procedural”, usam ifs em cascata e variáveis globais.

P3 – CERVEJARIA



1 Perceber herança



- Confundem “é um” com “tem um”.
- Ex.: Cerveja é um Item (não tem um Item).

2 Encapsulamento



- Atributos públicos.
- Alteração direta do id.
- Ausência de getters/setters.
- Não percebem porquê.

3 IDs sequenciais estáticos



- Dificuldade com atributos estáticos (ex.: `private static int proxId`).
- Confundem atributo da classe com atributo do objeto.

4 Regras de negócio



- Francesinha vegetariana.
- Entrega de acompanhamento apenas após Francesinha.
- Não sabem onde colocar as validações (colocam no main).

5 Uso de coleções



- Percorrer listas.
- Remover elementos.
- Procurar por id.
- Evitar duplicados.

6 instanceof



- Decoram a sintaxe, mas não entendem o conceito de polimorfismo por trás.

P7 – LOJA INFORMÁTICA



1 Compatibilidade entre objetos



- Dificuldade em validar relações entre componentes.
- Ex.: AM5 → DDR5.
- Comparar atributos entre objetos.

2 Estado do sistema



- Memória só pode ser vendida após uma PlacaMae compatível.
- Implica guardar histórico e lembrar a última venda (estado interno).

3 Separação de responsabilidades



- Lógica espalhada.
- Validações no main.
- Código duplicado.
- Não percebem que cada classe deve proteger o seu estado.

4 Polimorfismo



- Guardam tudo como `ArrayList<Componente>` e não percebem porquê funciona e o que isso representa.

5 Estruturação do código



- Métodos gigantes.
- Repetição de código.
- Falta de reutilização.
- Baixa coesão e legibilidade.

P10 – LOJA INFORMÁTICA



(VERSÃO AVANÇADA)

1 Pensar em objetos e não em passos



- Os melhores modelam entidades.
- Os restantes tentam “fazer funcionar”.
- Diferença enorme na qualidade do código.

2 Estado e histórico



- Grande dificuldade em guardar o último componente vendido.
- Validar compatibilidades futuras.
- Esquecem que o sistema tem memória.

3 Abstração



- Dificuldade em perceber superclasse e subclasses.
- Repetem atributos e métodos nas subclasses.

4 Onde colocar validações



- Construtor?
- `addStock()`?
- `vender()`?
- `main()`?
- Não sabem onde a regra faz mais sentido existir.

5 Polimorfismo + instanceof

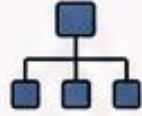


- Uso excessivo de `instanceof`.
- Não usam métodos polimórficos.
- Resoem tudo com `if / else if / else if ...` (código procedural).

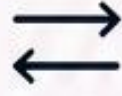
DIFICULDADES TRANSVERSAIS ÀS 3 TURMAS



Encapsulamento



Herança



Polimorfismo



Listas / Coleções



Remoção segura
de elementos



Atributos
estáticos



Organização
de classes

DIFICULDADES CONCEPTUAIS TRANSVERSAIS



Modelação
orientada a objetos



Regras de
negócio



Responsabilidade
das classes



Estado interno
do sistema



Validação
consistente

A MAIOR DIFICULDADE REAL



“escrever instruções”
(modo procedural)



**“modelar entidades
com comportamento”**
(pensamento orientado a objetos)



Quando o aluno percebe esta transição, a qualidade do código sobe imediatamente.
Quando não percebe... nasce mais um `main()` com 300 linhas e 17 ifs encaixados. A fauna habitual da programação universitária.

A maior dificuldade não é Java.
É a transição mental:

```
“escrever instruções”  
↓  
“modelar entidades com comportamento”
```



TURMA P3 – SISTEMA DE GESTÃO DE UMA CERVEJARIA



Objetivo:

Relembrar conceitos chave de POO aplicados ao sistema da cervejaria.

PERGUNTAS E RESPOSTAS

1



O que representa a classe Item no sistema?

Resposta:

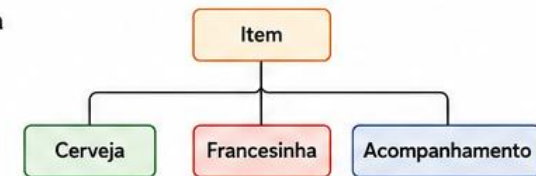
É a classe pai (superclasse) que representa um item genérico da cervejaria. Contém os atributos comuns a todos os itens: nome, preço base e id.

2

Qual a diferença entre composição e herança neste sistema?

Resposta:

É usada herança. Por exemplo, Cerveja, Francesinha e Acompanhamento “são” Items (herdam de Item) e não “têm” um Item (composição).



3



Porque deve o atributo id ser privado?

Resposta:

Para proteger o estado do objeto. Se fosse público, outra classe poderia alterá-lo livremente, quebrando a integridade do sistema (ex.: ids duplicados ou fora de ordem).

4

Como garantimos ids sequenciais a começar em 10?

Resposta:

Usa-se um atributo estático na classe Item que vai sendo incrementado sempre que é criado um novo item.

```
public abstract class Item {  
    private static int proxId = 10;  
    private int id;  
  
    protected Item(String nome, double preco) {  
        this.id = proxId++;  
        this.nome = nome;  
        this.precoBase = preco;  
    }  
}
```

5



Quando é que uma Francesinha é vegetariana?

Resposta:

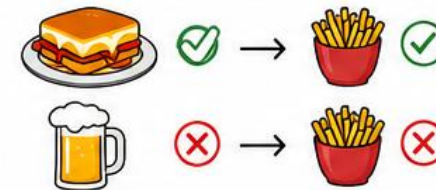
Quando o atributo vegetariana é true. Nesse caso, o tipo da carne tem de ser “vegetariana”. Mesmo que tenha bacon, o tipo da carne tem de ser compatível.

6

Qual a restrição na entrega de Acompanhamentos?

Resposta:

Só é possível entregar um Acompanhamento se o item vendido imediatamente antes for uma Francesinha. Caso contrário, a entrega deve ser rejeitada.



7



O que faz o método fazer(Item i) na Cervejaria?

Resposta:

Adiciona o item à fila (stock) de itens disponíveis para entrega, desde que não exista outro item com o mesmo id.

8

Qual a função do método entregar(int id, double preco)?

Resposta:

Remove o item com o id indicado da fila, regista a venda pelo preço fornecido e calcula o lucro (preço - preço base). Devolve true se a entrega foi feita, false caso contrário.

```
public boolean entregar(int id, double preco) {  
    // verifica disponibilidade  
    // verifica regras de negócio  
    // regista lucro  
    // remove da fila  
}
```

9



Como calculamos o lucro total da cervejaria?

Resposta:

Acumulamos, para cada item entregue, a diferença entre o preço de venda e o preço base de produção.

10

Como listamos apenas os Acompanhamentos disponíveis?

Resposta:

Percorremos a fila de itens e usamos instanceof Acompanhamento para imprimir apenas os que são desse tipo.

```
if (item instanceof Acompanhamento) {  
    System.out.println(item);  
}
```



DICAS DE OURO

- **Promova o encapsulamento:** mantenha atributos privados e use getters quando necessário.
- Valide regras de negócio dentro dos métodos (fazer, entregar).
- Use **herança** para evitar código repetido.
- Pense no **estado do sistema**: o que aconteceu antes pode condicionar o que pode acontecer agora.

CONCEITOS CHAVE ABORDADOS



Herança



Encapsulamento



Construtores



Polimorfismo



Coleções
(Lista/Fila)



Regras de
Negócio



Estado e
Histórico



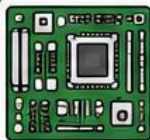
TURMA P7 – LOJA INFORMÁTICA

Perguntas de bolso + Respostas



Objetivo: Relembrar conceitos chave de POO e as regras de negócio da loja informática.

1



Qual é a classe base do sistema e o que representa?

Resposta:

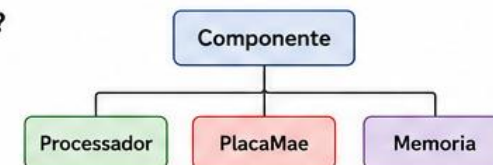
A classe base é Componente. Representa um componente genérico da loja, com atributos comuns: nome, marca, preço de aquisição e id.

2

Quais são as subclasses de Componente?

Resposta:

Processador, PlacaMae e Memoria são subclasses de Componente e adicionam atributos específicos a cada tipo de componente.



3



Porque deve o atributo id ser privado?

Resposta:

Para proteger o estado do objeto e evitar alterações indevidas. Garante integridade (ex.: não permitir id duplicados ou fora de ordem).

4

Como garantimos ids sequenciais a começar em 100?

Resposta:

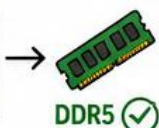
Usa-se um atributo estático na classe Componente que é incrementado sempre que é criado um novo componente.

```

public abstract class Componente {
    private static int proxId = 100;
    private int id;

    protected Componente(String nome, String marca, double preco) {
        this.id = proxId++;
        this.nome = nome;
        this.marca = marca;
        this.precoBase = preco;
    }
}
  
```

5



Qual é a restrição entre PlacaMae e Memoria?

Resposta:

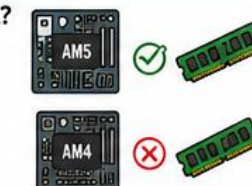
Se a PlacaMae for AM5, o tipo de memória tem de ser DDR5. Esta validação deve ser feita (ex.: no construtor da PlacaMae ou no método addStock).

6

Quando é que podemos vender uma Memoria?

Resposta:

Só podemos vender Memoria se o componente vendido imediatamente antes for uma PlacaMae compatível (mesmo tipo de RAM e mesmo tipo de slot: portátil ou não).



Compatível se:

- ✓ mesmo tipo de RAM
- ✓ mesmo tipo de slot (portátil ou não)

7



O que faz o método addStock(Componente c)?

Resposta:

Adiciona o componente ao stock da loja. Não permite duplicados: se já existir um componente com o mesmo id, não é adicionado.

8

Qual é o objetivo do método vender(int id, double preco)?

Resposta:

Vender o componente com o id indicado, pelo preço fornecido. Remove do stock, regista o lucro (preco – precoBase) e devolve true se a venda for feita, false caso contrário.

```

public boolean vender(int id, double preco) {
    // verifica disponibilidade
    // verifica regras de negócio
    // regista lucro e remove do stock
}
  
```

9



Como calculamos o lucro total da loja?

Resposta:

Somamos, para cada componente vendido, a diferença entre o preço de venda e o preço base (preço de aquisição).

10

Qual é a diferença entre imprimir todos e imprimir apenas memórias?

Resposta:

imprimeTodos() percorre todo o stock e imprime todos os componentes. imprimeMemorias() usa instanceof Memoria para imprimir apenas os que são do tipo Memoria.

```

for (Componente c : stock) {
    if (c instanceof Memoria) {
        System.out.println(c);
    }
}
  
```



DICAS DE OURO

- Promova o encapsulamento: atributos privados + getters quando necessário.
- Valide regras de negócio sempre que possível no ponto mais próximo do problema.
- Guarde o último componente vendido para aplicar regras de venda condicional.
- Evite código repetido: reutilize métodos e promova polimorfismo.

CONCEITOS CHAVE ABORDADOS



Herança



Encapsulamento



Construtores



Polimorfismo



Coleções
(Lista)



Regras de
Negócio



Estado e
Histórico

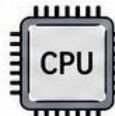


TURMA P10 – LOJA INFORMATICA (versão avançada)

Perguntas de bolso + Respostas



Objetivo: Relembrar conceitos chave de POO aplicados à gestão avançada de componentes e às regras de negócio da loja informática.

1  Qual é a classe base do sistema e o que representa?

Resposta:
A classe base é Componente.
Representa um componente genérico da loja, com atributos comuns: nome, marca, preço de aquisição e id.

2 Quais são as subclasses de Componente?

Resposta:
Processador, PlacaMae e Memoria são subclasses de Componente e adicionam atributos específicos a cada tipo de componente.

```

graph TD
    Componente --> Processador
    Componente --> PlacaMae
    Componente --> Memoria
  
```

3  Porque deve o atributo id ser privado?

Resposta:
Para proteger o estado do objeto. Se fosse público, outra classe poderia alterá-lo livremente, quebrando a integridade do sistema (ex.: ids duplicados ou fora de ordem).

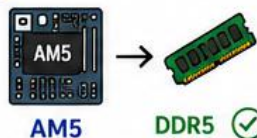
4 Como garantimos ids sequenciais a começar em 200?

Resposta:
Usa-se um atributo estático na classe Componente inicializado a 199 e incrementado sempre que é criado um novo componente.

```

public abstract class Componente {
    private static int proxId = 199;
    private int id;

    protected Componente(String nome, String marca, double preco) {
        this.id = ++proxId;
        this.nome = nome;
        this.marca = marca;
        this.precoBase = preco;
    }
}
  
```

5  Qual é a restrição entre PlacaMae e Memoria?

Resposta:
Se a PlacaMae for AM5, o tipo de memória tem de ser DDR5. Esta validação deve ser feita (ex.: no construtor da PlacaMae ou no método addStock).

6 Quando é que podemos vender uma Memoria?

Resposta:
Só podemos vender Memoria se o componente vendido imediatamente antes for uma PlacaMae compatível (mesmo tipo de RAM e mesmo tipo de slot: portátil ou não).

Compatível se:

- ✓ mesmo tipo de RAM (DDR4 ou DDR5)
- ✓ mesmo tipo de slot (portátil ou não)

7  O que faz o método addStock(Componente c)?

Resposta:
Adiciona o componente ao stock da loja.
Não permite duplicados: se já existir um componente com o mesmo id, não é adicionado.

8 Qual é o objetivo do método vender(int id, double preco)?

Resposta:
Vender o componente com o id indicado, pelo preço fornecido. Remove do stock, regista o lucro (preco - precoBase) e devolve true se a venda for feita, false caso contrário.

```

public boolean vender(int id, double preco) {
    // verifica disponibilidade
    // verifica regras de negócio
    // regista lucro e remove do stock
}
  
```

9  Como calculamos o lucro total da loja?

Resposta:
Somamos, para cada componente vendido, a diferença entre o preço de venda e o preço base (preço de aquisição).

10 Qual é a diferença entre imprimeTodos() e imprimeMemorias()?

Resposta:
imprimeTodos() percorre todo o stock e imprime todos os componentes. imprimeMemorias() usa instanceof Memoria para imprimir apenas os componentes do tipo Memoria.

```

for (Componente c : stock) {
    if (c instanceof Memoria) {
        System.out.println(c);
    }
}
  
```



DICAS DE OURO

- Promova o **encapsulamento**: atributos privados + getters quando necessário.
- Valide regras de negócio sempre que possível no ponto mais próximo do problema.
- Guarde o **último componente vendido** para aplicar regras de venda condicional.
- Use **herança** para evitar código repetido e tirar partido do polimorfismo.
- Pense no **estado do sistema**: o que aconteceu antes pode condicionar o que pode acontecer agora.

CONCEITOS CHAVE ABORDADOS



Herança



Encapsulamento



Construtores



Polimorfismo



Coleções (Lista)



Regras de Negócio



Estado e Histórico



PERGUNTA FINAL – PARA PENSAR E FECHAR O TEMA



Qual das seguintes abordagens está mais alinhada com Programação Orientada a Objetos?
Escolhe a melhor opção.

TIPO DE RESPOSTA

- ✓ Escolha única
- Múltipla escolha
- ✗ Verdadeiro/Falso

OPÇÃO A – ABORDAGEM PROCEDURAL

```
if(tipo.equals("memoria")) {  
    ...  
}  
else if(tipo.equals("processador")) {  
    ...  
}  
else if(tipo.equals("placamae")) {  
    ...  
}
```



- Pensa em “tipos” e instruções.
- Centraliza decisões.
- Cresce mal (muitos ifs).
- Código procedural.

OPÇÃO B – ABORDAGEM ORIENTADA A OBJETOS

```
Componente c = new Memoria(...);  
c.imprimir();
```



- Delega comportamento ao objeto.
- Cada objeto sabe o que fazer.
- Usa polimorfismo.
- Menos ifs e mais extensível.

VS

RESPOSTA CORRETA

DISCUSSÃO CONCEITUAL

A opção A (procedural):

- Usa condições para decidir o que fazer.
- O código cresce e fica difícil de manter.
- Mistura responsabilidades.



A opção B (POO):

- Cada objeto é responsável pelo seu comportamento.
- Aproveita polimorfismo.
- Código mais limpo, flexível e reutilizável.

A GRANDE TRANSIÇÃO



“PENSAR EM INSTRUÇÕES”

(modo procedural)

Perguntamos:

“O que devo fazer agora?”

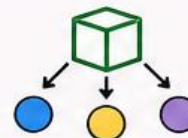


“PENSAR EM OBJETOS”

(modo orientado a objetos)

Perguntamos:

“Quem deve ser responsável por isto?”



FRASE FINAL

“Na programação procedural perguntamos: ‘o que devo fazer agora?’
Em POO perguntamos: ‘quem deve ser responsável por isto?’”



LEMBRA-TE

Programar não é apenas escrever código.
É modelar entidades com comportamento
e responsabilidades bem definidas.



Quando percebes esta transição, a qualidade do teu código sobe imediatamente!

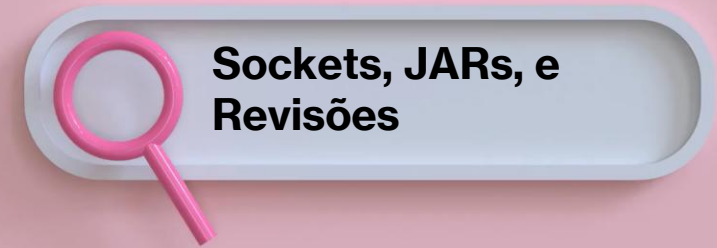
Quando não percebes... nascem mais métodos main() com 300 linhas e 17 ifs encaixados. 😊

Os objectivos desta aula são:

- Comunicação através da rede.
- Reutilização de padrões de código para I/O.
- Criação de JARs e uso de parâmetros na linha de comandos.
- Revisões para o mini-teste 2.

Tópicos

- ServerSocket e Socket.
- InputStream/OutputStream utilizando Sockets.
- Empacotamento em JAR.



Exercícios

Exercício 11.1: Comunicação em Rede com Sockets

Exercício 11.1 a) e b): Construção de um Echo Server e Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

Exercício 11.2: Distribuição e Parametrização de Aplicações

- Empacotamento em Arquivos JAR e Argumentos de Linha de Comandos

Exercício 11.3: Gestão de Encomendas e Persistência de Dados

- Desenvolvimento do Parcel Manager com Java Collections e Fluxos de I/O



Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) e b): Construção de um Echo Server e Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

Um **Echo Server** é um serviço de rede simples onde o servidor repete para o cliente todas as mensagens recebidas (i.e., com se fosse uma voz de eco).

A **classe Socket** tem os métodos `getInputStream()` e `getOutputStream()` que permite comunicar através da rede utilizando as mesmas abordagens de leitura e escrita já utilizadas em ficheiros (pex., `Scanner` ou `PrintWriter`).

- a) Crie um Echo Server simples utilizando a classe `SocketServer`.
- b) Crie um cliente que interage com um echo server, e teste ambos os programas.
- c) Utilizando como base o programa de cliente, crie um novo utilitário para estimar o Round-Trip-Time (RTT) de uma ligação.

Exercício 11.1 – Comunicação Cliente/Servidor em Java

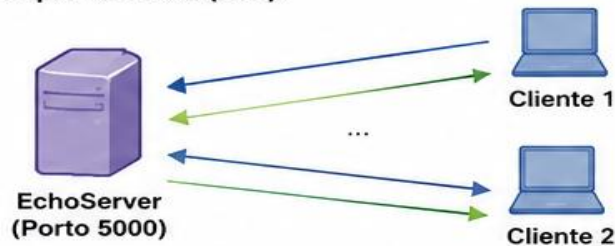
Objectivo: Implementar um Echo Server e clientes em Java.
O cliente Eco envia mensagens e recebe o eco do servidor.
O cliente RTT estima o Round-Trip Time (RTT).

Legenda

-  mensagem do cliente para o servidor
-  resposta do servidor para o cliente

a) EchoServer

Servidor que fica à escuta num porto, recebe mensagens e devolve exatamente o que recebeu (eco).



Funcionamento do servidor

1. Cria ServerSocket(5000)
2. Espera por ligação de um cliente
3. Lê uma mensagem (readLine)
4. Devolve a mesma mensagem (println)
5. Se receber "sair", fecha a ligação
6. Volta ao passo 2 (atende outro cliente)

Exemplo (servidor recebe e devolve)

Recebido: Olá servidor!
Enviado: Olá servidor!

b) EchoClient

Cliente interativo que liga ao servidor, envia mensagens escritas pelo utilizador e mostra o eco recebido.



Funcionamento do cliente (Eco)

1. Liga ao servidor (Socket)
2. Lê texto do teclado (utilizador)
3. Envia mensagem ao servidor (println)
4. Recebe eco do servidor (readLine)
5. Mostra a resposta no ecrã
6. Repete até o utilizador escrever "sair"

Exemplo (execução do EchoClient)

```
> Olá servidor!  
Eco do servidor: Olá servidor!  
> Como estás?  
Eco do servidor: Como estás?  
> sair  
Ligação terminada.
```

c) RTTClient

Cliente que estima o Round-Trip Time (RTT). Envia o timestamp atual, recebe o eco do servidor e calcula o tempo de ida e volta.



Passos do RTTClient (para cada medição)

1. tempoEnvio = System.nanoTime()
2. Envia tempoEnvio ao servidor
3. Recebe o tempo (eco do servidor)
4. tempoRececao = System.nanoTime()
5. $RTT = tempoRececao - tempoEnvio$
6. Mostra o RTT (em ms)
7. Repete 10 vezes (ex.: com pausa de 1s)

Exemplo (execução do RTTClient)

Ligado ao servidor.
RTT = 0,721 ms
RTT = 0,666 ms
...
RTT = 0,728 ms
(são 10 medições)

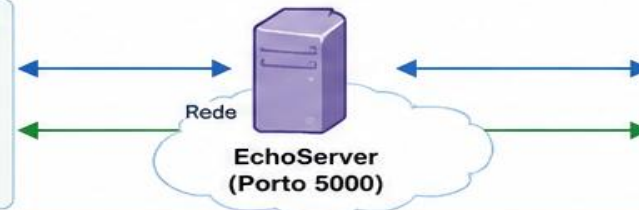
Como tudo funciona em conjunto

1) Iniciar o servidor

```
$ java EchoServer  
Servidor à escuta  
no porto 5000...
```

2) Ligar um cliente Eco (opcional)

```
$ java EchoClient  
> Olá  
Eco do servidor: Olá  
> sair  
Ligação terminada.
```



3) Ligar o cliente RTT

```
$ java RTTClient  
Ligado ao servidor.  
RTT = 0,721 ms  
...  
RTT = 0,728 ms
```

Notas importantes

- O EchoServer pode atender vários clientes, um de cada vez.
- O RTT mede apenas o tempo de ida e volta da mensagem (latência).
- Usamos System.nanoTime() para maior precisão temporal.



A i n t e r n e t

A comunicação na internet é baseada num modelo chamado client/server (cliente/servidor).

O cliente é o dispositivo que está a consumir os serviços disponíveis na rede.

Na outra ponta está o servidor, que é o dispositivo que hospeda estes serviços.

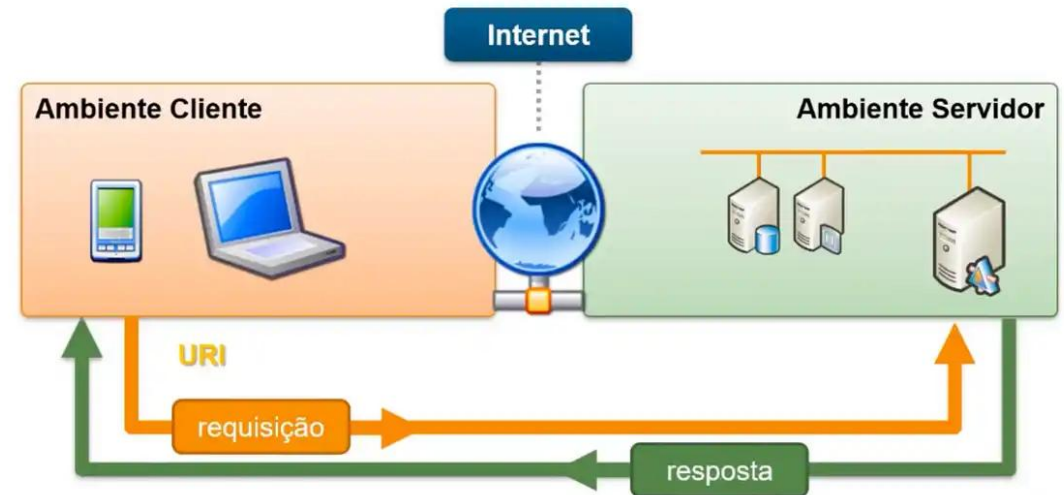
Dizer que um sítio da internet está hospedado significa dizer que ele está num servidor, conectado 24/7 à internet.

Carregando em “enter” depois de digitar uma busca ou um endereço no navegador, essa acção transforma-se numa requisição e trafega na rede até achar o servidor que atende essa solicitação.

| Redes: O Modelo Cliente-Servidor

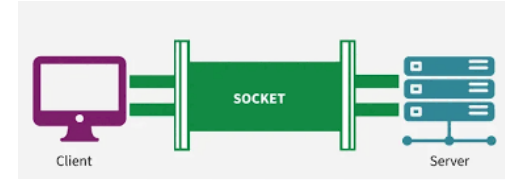
A comunicação em rede baseia-se num diálogo estruturado entre dois agentes:

- **Servidor:** Um serviço que "escuta" pedidos num porto (Port) específico de um endereço IP.
- **Cliente:** Um processo que inicia a ligação ao servidor para solicitar informação ou ação.
- **Protocolo:** O conjunto de regras (ex: TCP) que define como os dados são transmitidos sem erros.



<https://www.dio.me/articles/client-x-server-entendendo-a-comunicacao-na-internet>

| Sockets: Endpoints de Comunicação



Server Socket (A "Central Telefónica"): É um objecto que não serve para falar nem para ouvir dados. A sua única função na vida é reservar um Porto (Port) na máquina e ficar em estado de escuta (listening). Ele espera que um cliente tente ligar-se.

Socket (O "Telefone de Linha Directa"): Quando o ServerSocket aceita uma ligação através do método `.accept()`, ele cria e devolve um objecto Socket normal. Este Socket representa o canal exclusivo e dedicado para comunicar com aquele cliente específico que acabou de ligar.

ServerSocket

Utilizado no lado do Servidor. Fica em estado de "escuta" (listening) até que uma tentativa de conexão seja detetada. Ao aceitar, cria um Socket dedicado para essa sessão.

Socket

Utilizado por ambos. Representa o canal de comunicação ativo. Através dele, podemos obter fluxos (Streams) para ler e escrever dados como se fosse um ficheiro local.

Um socket é o "endpoint" da comunicação. Quando o Cliente se liga ao Servidor, é criada uma "tomada" (Socket) em cada ponta. Esta "ligação" é como um cano invisível entre os dois computadores.

| Portos de rede



Internet Assigned Numbers Authority
<https://www.internetsociety.org/>

Os portos dividem-se em três grandes intervalos regulados pela IANA (Internet Assigned Numbers Authority):

- 0 a 1023
 - Portos Bem Conhecidos (Well-Known)
 - Reservados para o sistema operativo e serviços essenciais do sistema (ex: porto 80 para HTTP, porto 443 para HTTPS, porto 22 para SSH).
 - Para abrir um servidor nestes portos, o utilizador precisa de privilégios de Administrador (root).
- 1024 a 49151
 - Portos Registados (Registered)
 - Livres para aplicações comuns e serviços de terceiros (ex: o porto 1433 é usado pelo SQL Server, o porto 3306 pelo MySQL).
 - Qualquer utilizador comum pode abrir um programa nestes portos.
- 49152 a 65535
 - Portos Dinâmicos/Efémeros
 - Usados temporariamente pelo sistema operativo quando o Cliente inicia uma ligação.
 - São os portos gerados aleatoriamente para servir de "remetente" na comunicação.

NETWORK PORT

Well-Known Ports

0-1023

Registered Ports

1024-49151

Dynamic Port

49152-65565

<https://www.geeksforgeeks.org/computer-networks/difference-between-ip-address-and-port-number/>

| Categorias e Disponibilidade de Portos de Comunicação



serverSocket.accept():

Este método suspende a execução da *thread* principal. O servidor não gasta processamento desnecessário enquanto espera; fica adormecido pelo sistema operativo até que um pacote de rede TCP de pedido de ligação (*SYN*) chegue ao porto 5000.

InputStreamReader:

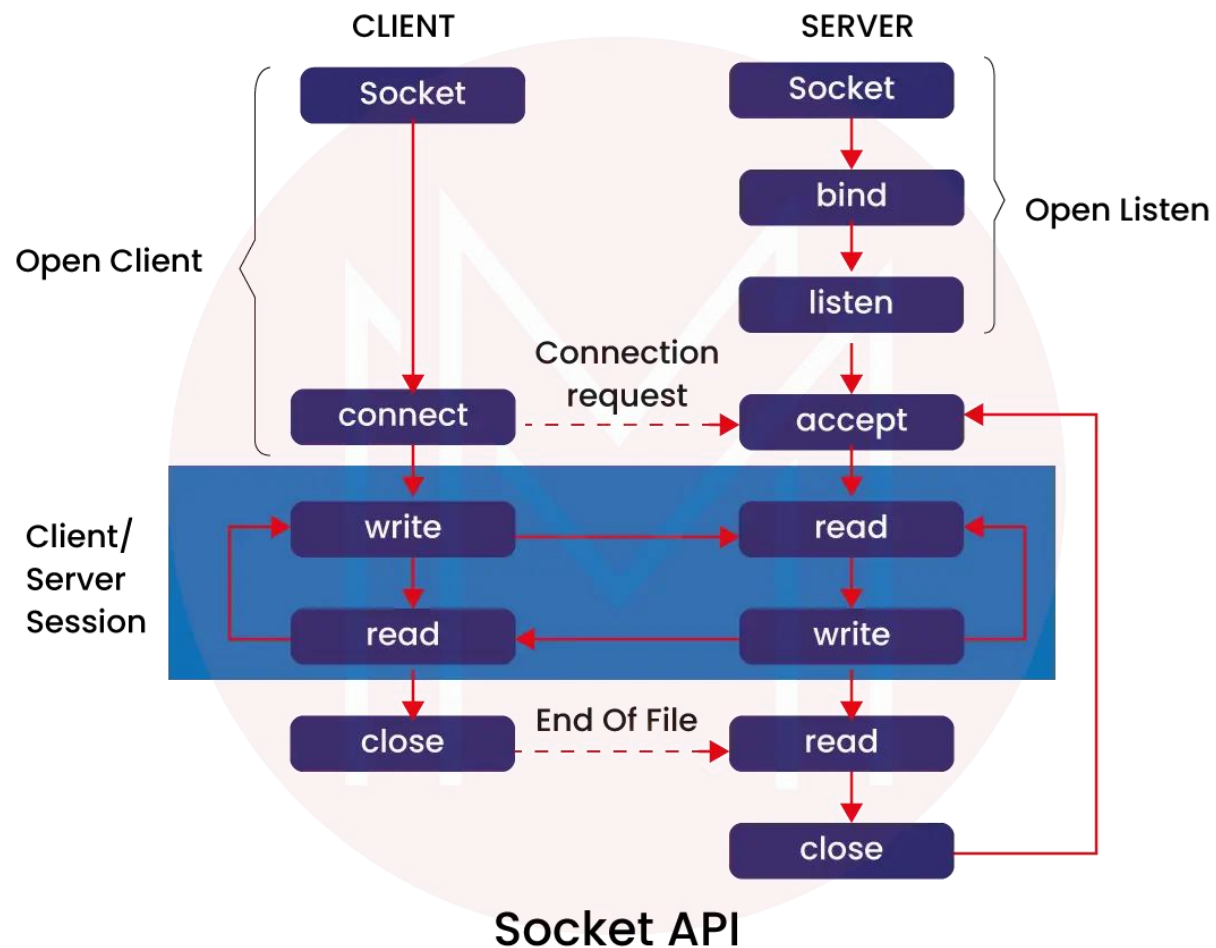
O método `clientSocket.getInputStream()` devolve um fluxo de bytes brutos (`byte[]`). Como estamos a comunicar em formato de texto (`Strings`), o `InputStreamReader` actua como uma ponte que traduz esses bytes em caracteres.

BufferedReader:

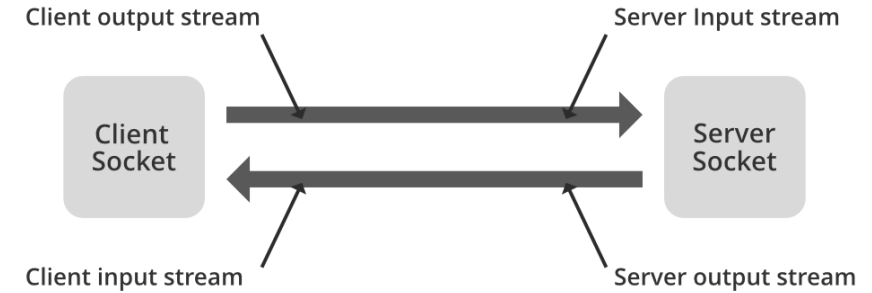
Envolve o leitor de caracteres para criar um *buffer* de memória. Isto permite-nos utilizar o método extremamente conveniente `readLine()`, que lê o fluxo continuamente até encontrar uma quebra de linha (`\n` ou `\r\n`), poupando acessos directos ao *hardware* da rede.

autoFlush (true):

Na rede, os dados acumulam-se num *buffer* de saída antes de serem disparados. Se não fizéssemos o *flush*, o eco poderia ficar retido na memória do servidor e o cliente ficaria eternamente à espera. O `true` garante o envio imediato após cada `println`.



| Fluxos de Dados (Streams)



InputStream

Lê os dados que chegam da rede. Frequentemente encapsulado num Scanner para facilitar a leitura de texto.

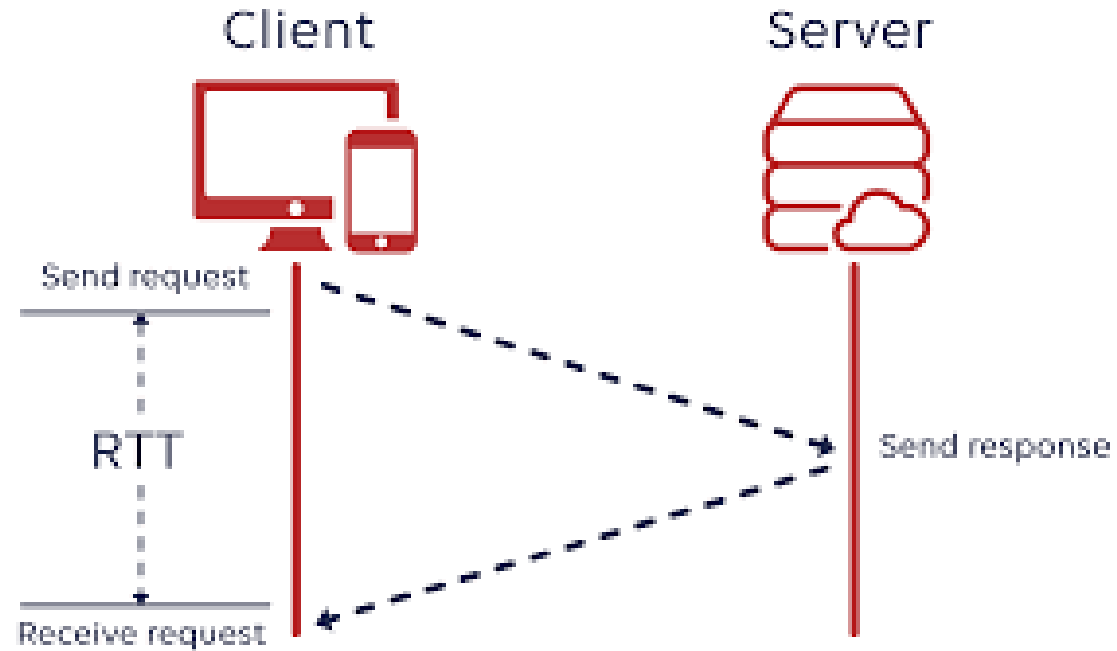


OutputStream

Envia os dados para o outro extremo. Geralmente usado com PrintWriter para enviar strings formatadas.

Streams: São os canais que permitem que a água (os dados) flua dentro do cano. **InputStream (Canal de Entrada)**: É por onde o programa **recebe** os dados que vêm da rede. Imagina que é o "ouvido". **OutputStream (Canal de Saída)**: É por onde o teu programa **envia** os dados para a rede. É a tua "boca".

| Latência: Round-Trip Time (RTT)



O que mede o RTT?

É o tempo que uma mensagem demora a ir da origem ao destino e a resposta a voltar.

$$RTT = T_{\text{chegada}} - T_{\text{envio}}$$

Essencial para medir a qualidade e a velocidade de uma ligação de rede.

| Empacotamento: Ficheiros JAR

Java Archive (JAR) é um formato de ficheiro comprimido (baseado em ZIP) usado para agrupar múltiplas classes Java e recursos.

- **Portabilidade:** Distribui toda a aplicação num único ficheiro.
- **Manifest:** Define qual a classe que contém o método `main`.
- **Eficiência:** Reduz o espaço ocupado e facilita a organização do projeto binário.



| Interface de Linha de Comandos (CLI)

CLI (sigla para Command-Line Interface ou Interface de Linha de Comando) é um mecanismo que permite a interação com o computador exclusivamente através de textos e comandos digitados

Main Arguments

O parâmetro `String[] args` do método `main` permite que o utilizador passe dados para o programa no momento do arranque (ex: IP do servidor).

// Exemplo de Execução

```
java -jar app.jar localhost 8080
```

```
args[0] = "localhost"
```

```
args[1] = "8080"
```



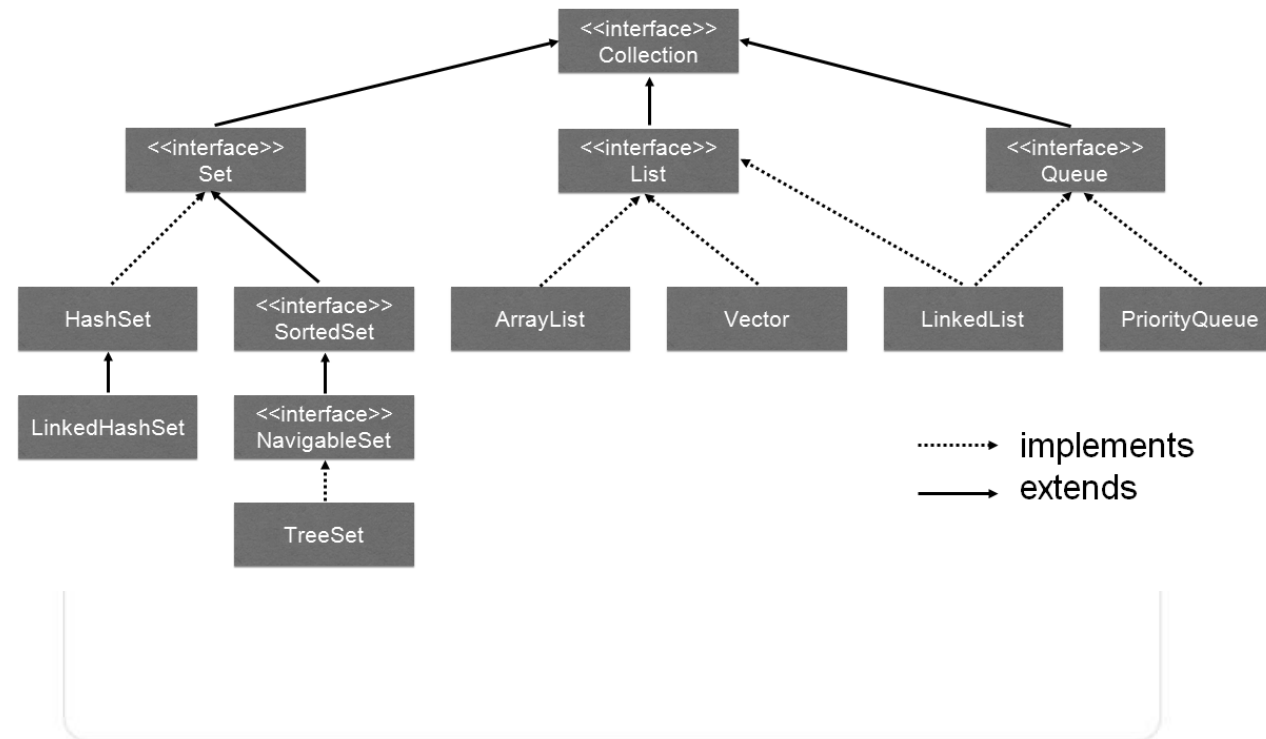
| Java Collections Framework

O Java Collections Framework (JCF) é uma arquitetura padrão que fornece classes e interfaces para armazenar e manipular grupos de objetos como uma única unidade

Um conjunto de interfaces e classes que permitem gerir grupos de objetos de forma otimizada.

- **Lists:** Coleções ordenadas (permitem duplicados).
- **Maps:** Mapeamento de chaves para valores (procura rápida).
- **Sets:** Conjuntos sem elementos duplicados.

Collection Interface



| Persistência e Parsing de Dados

O parsing e a persistência são etapas críticas onde dados são lidos/interpretados e salvos em sistemas de armazenamento. Erros nessas fases exigem tratamentos específicos (exceções) para evitar corrupção de dados ou falhas no sistema.



Parsing (split)

Processo de dividir uma linha de texto em partes úteis usando separadores (como o ";").



Persistência

Guardar o estado da aplicação em ficheiros para que os dados não se percam ao fechar.



Exceções

Tratamento de erros obrigatório (IOException) ao lidar com ficheiros ou rede.



Exercício 11.1a)

Comunicação em Rede com Sockets EchoServer

Exercício 11.1 (a) – EchoServer

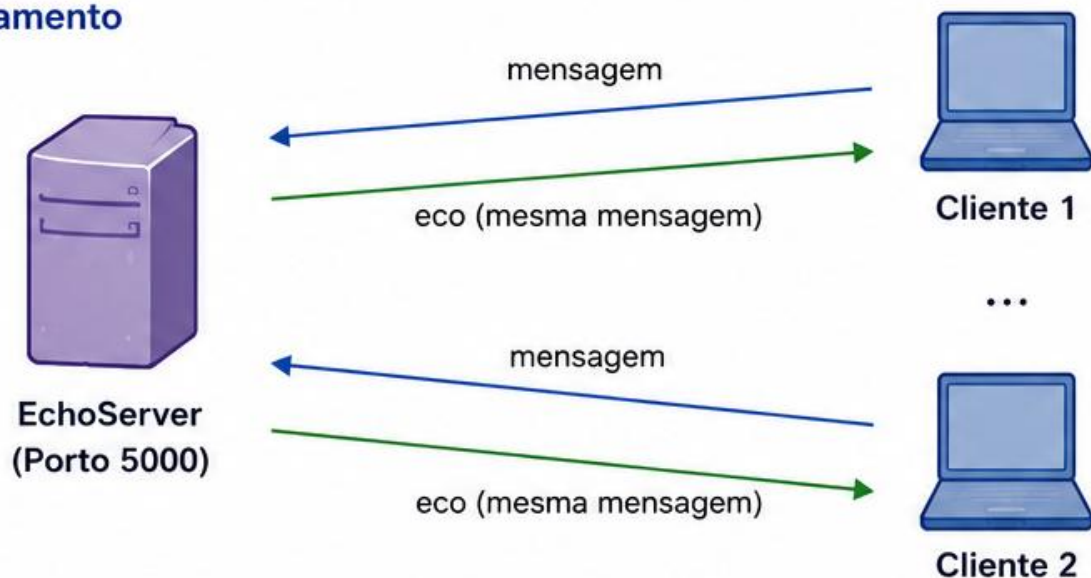
Objetivo Criar um servidor (Echo Server) que:

- fica à escuta num porto;
- recebe mensagens dos clientes;
- devolve exatamente a mesma mensagem (eco).

Ideia principal

O servidor espera por ligações de clientes.
Para cada mensagem recebida, o servidor devolve exatamente a mesma mensagem.

Funcionamento



Passos do EchoServer

1. Cria um ServerSocket (ex.: porto 5000)
2. Espera por ligação de um cliente (accept)
3. Lê uma mensagem (readLine)
4. Devolve a mesma mensagem (println)
5. Se receber "sair", fecha a ligação
6. Volta ao passo 2 (atende outro cliente)

Exemplo (interação)

Cliente: Olá servidor!

Servidor: Olá servidor!

Cliente: Como estás?

Servidor: Como estás!

Cliente: sair

Servidor: sair → ligação terminada.

Legenda

-  mensagem do cliente para o servidor
-  resposta do servidor para o cliente

Classes principais utilizadas

- ServerSocket – cria o servidor e fica à escuta
- Socket – representa a ligação com o cliente
- BufferedReader – lê mensagens do cliente
- PrintWriter – envia mensagens (eco) ao cliente

Estrutura de Classes (java.io & java.net)

01

Classes do Pacote java.net (Comunicação em Rede)

- **ServerSocket**: Utilizada para criar o socket do servidor que escuta numa porta específica (5000) à espera de conexões vindas dos clientes.
- **Socket**: Representa a conexão propriamente dita entre o servidor e o cliente após o método accept() ser bem-sucedido.

02

Classes do Pacote java.io (Entrada e Saída de Dados)

BufferedReader: Utilizada para ler texto de um fluxo de entrada de forma eficiente, uso do método readLine().

- **InputStreamReader**: ponte entre fluxos de bytes e fluxos de caracteres. Lê bytes do socket e traduz-os em caracteres.
- **PrintWriter**: formatar e enviar representações de texto para o fluxo de saída do socket (enviando a resposta de volta ao cliente com println).
- **IOException** captura qualquer falha que possa ocorrer nas operações de entrada/saída ou de rede (como uma porta já ocupada ou uma desconexão abrupta).

03

Classes do Pacote java.lang (Invocadas Implicitamente)

- System**: Utilizada através de System.out e System.err para interagir com as saídas standard do sistema (consola).
- **String**: Utilizada para manipular a variável line e os argumentos do método main.

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conex?o de um cliente...
```

```
package aula11;
```

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.io.IOException;
```

```
import java.net.ServerSocket;
import java.net.Socket;
```

```
import java.io.*;
import java.net.*;
```

1.Importações

- `package aula11;`: Define que esta classe pertence ao pacote `aula11`, organizando a directoria do projecto.
- `java.io.*`: Importa as classes de Input/Output necessárias para ler e escrever texto
- `java.net.*`: Importa as ferramentas de rede essenciais (`ServerSocket` e `Socket`).

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
```

A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conex?o de um cliente...

```
package aula11;
```

```
import java.io.BufferedReader;  
import java.io.InputStreamReader;  
import java.io.PrintWriter;  
import java.io.IOException;  
import java.net.ServerSocket;  
import java.net.Socket;
```

```
public class Echoserver {
```

```
    public static void main(String[] args)  
    {  
        int port = 5000; // Porto padrão  
        onde o servidor vai escutar
```

```
        System.out.println("A iniciar o  
Echo Server no porto " + port + "...");  
        //feedback visual ao utilizador
```

2. Configuração inicial

- `int port = 5000;`: Define a extensão telefónica da máquina.

Nota: O número 5000 é escolhido porque portos abaixo de 1024 são reservados para o sistema operativo (como o porto 80 para HTTP). Qualquer número inteiro entre 1024 e 65535 está totalmente livre para ser utilizado

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conex?o de um cliente...
```

3. O Primeiro Try (A Central Telefónica)

- `new ServerSocket(port)`: faz o registo junto do sistema operativo. A partir daqui, o porto 5000 fica ocupado.
- `try (...)`: garante que, quando o bloco terminar (ou se houver um erro de rede), o método `.close()` do `ServerSocket` é chamado nos bastidores de forma invisível. Isto liberta o porto imediatamente, evitando o erro de "Porto já em uso".

```
package aula11;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Echoserver {
    public static void main(String[] args) {
        int port = 5000; // Porto padrão onde o
        servidor vai escutar

        System.out.println("A iniciar o Echo
        Server no porto " + port + "...");
        //feedback visual ao utilizador

        // 1. Criar o ServerSocket dentro de um try-
        with-resources para garantir o fecho do
        porto
        try (ServerSocket serverSocket = new
        ServerSocket(port)) {
            System.out.println("Servidor em escuta.
            Aguardando conexão de um cliente...");
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
```

A iniciar o Echo Server no porto 5000...

Servidor em escuta. Aguardando conexão de um cliente...

```
// 1. Criar o ServerSocket dentro de um try-with-resources para garantir o fecho do porto
```

```
try (ServerSocket serverSocket = new ServerSocket(port)) {  
    System.out.println("Servidor em escuta. Aguardando conexão de um cliente...");
```

```
2. dentro de um try-with-resources aceita cliente, um canal de leitura e cria um canal de escrita
```

4. O Segundo Try (O Bloqueio e Aceitação)

- `serverSocket.accept()`: É aqui que o programa fica "à escuta".

O código congela nesta linha. Ele não passa para a instrução seguinte enquanto nenhum cliente iniciar uma conexão TCP para o porto 5000. Assim que um cliente se conecta, esta linha acorda e cria o `clientSocket`, que representa o canal de comunicação privado com o cliente. O método `.getRemoteSocketAddress()` imprime o IP e o porto gerado aleatoriamente pela máquina do cliente.

```
try (Socket clientSocket = serverSocket.accept());
```

```
BufferedReader in = new BufferedReader(new  
InputStreamReader(clientSocket.getInputStream()))  
;
```

```
PrintWriter out = new  
PrintWriter(clientSocket.getOutputStream(),  
true)) {
```

```
System.out.println("Cliente ligado.");
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
```

A iniciar o Echo Server no porto 5000...

Servidor em escuta. Aguardando conex?o de um cliente...

5. Configuração dos Fluxos (A Canalização)

- `clientSocket.getInputStream()`: Vai buscar o fluxo de bytes que entram pela placa de rede.
- `new InputStreamReader(...)`: Pega nos bytes e converte-os em caracteres legíveis.
- `new BufferedReader(...)`: Cria um reservatório de memória (buffer) que nos permite ler o texto linha a linha em vez de ler letra a letra, disponibilizando o método `.readLine()`.

// 2. O método `accept()` é BLOQUEANTE. O programa fica aqui parado até um cliente se ligar.

```
try (Socket clientSocket = serverSocket.accept())
{
    System.out.println("Cliente conectado com sucesso vindo de: " +
        clientSocket.getRemoteSocketAddress());
}
```

// 3. Obter os canais (Streams) de comunicação com este cliente específico
// Convertemos o InputStream bruto em texto através do BufferedReader

```
BufferedReader in = new BufferedReader(
    newInputStreamReader(clientSocket.getInputStream()))
```

// O PrintWriter facilita a escrita de texto para o OutputStream. O segundo parâmetro 'true' activa o "auto-flush", empurrando o texto imediatamente pela rede.

```
PrintWriter out = new
    PrintWriter(clientSocket.getOutputStream(), true);
```

```
String inputLine;
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



```
PS C:\P00\Exercicios_RMA> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\P00\Exercicios_RMA\bin' 'aula11.Echoserver'
```

A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conex?o de um cliente...

6. Ciclo de Leitura e o Eco

- `in.readLine()`: O código volta a congelar aqui, mas agora à espera que o cliente envie uma linha de texto.
- `!= null`: Se o cliente fechar a ligação, o fluxo termina e o `readLine()` devolve `null`, terminando o ciclo `while`.
- `out.println(inputLine);`: O coração do exercício. Pega na String exacta que acabou de ler do canal `in` e escreve-a de volta no canal `out`. O cliente recebe o seu próprio texto de volta.

```
// 4. Ciclo de leitura: Lê linha a linha o que o
cliente envia [cite: 20, 21]

while ((inputLine = in.readLine()) != null) {
    System.out.println("Recebido do cliente: " +
inputLine);

    // Se o cliente enviar uma palavra de ordem, podemos
    terminar a sessão
    if
("sair".equalsIgnoreCase(inputLine.trim())) {
        out.println("Adeus!");
        break;
    }

    // O ECO: Devolve exactamente a mesma string para o
    cliente [cite: 20]
    out.println(inputLine);
} // Aqui o clientSocket é fechado
automaticamente

System.out.println("Ligação com o cliente
terminada.");

} catch (IOException e) {
    System.err.println("Excepção capturada ao
tentar escutar no porto "
+ port + " ou ao
aguardar por uma ligação.");
    System.err.println(e.getMessage());
}
}
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

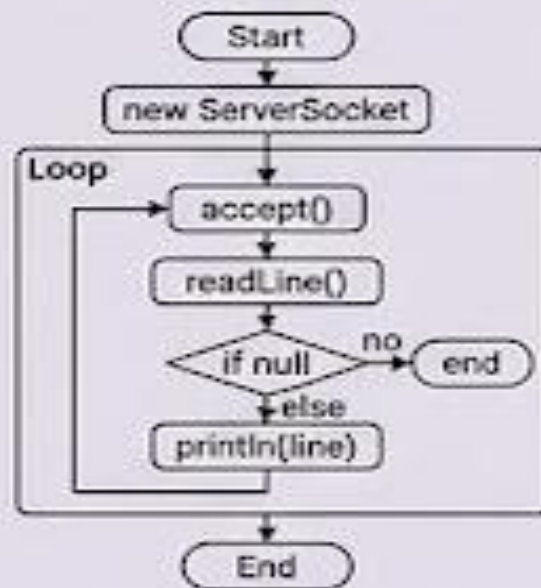
Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

EchoServer.java



A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conex?o de um cliente...

EchoServer Fluxo



```
package aula11;
import java.io.*;
import java.net.*;
public class EchoServer {

    public static void main(String[] args) {
        int port = 5000;
        System.out.println("A iniciar o Echo Server no
        porto " + port + "..."); //feedback visual ao
        utilizador

        try (ServerSocket serverSocket = new
        ServerSocket(port)) {

            System.out.println("Servidor em escuta. Aguardando
            conexao de um cliente...");

            try (Socket clientSocket = serverSocket.accept();

                BufferedReader in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));

                PrintWriter out = new
                PrintWriter(clientSocket.getOutputStream(), true)) {

                System.out.println("Cliente ligado.");
                String line;
                while ((line = in.readLine()) != null) {
                    out.println(line);
                }
            } catch (IOException e) {
                System.err.println("Erro: " +
                e.getMessage());
            }
        }
    }
}
```

A close-up, low-angle shot of a person's hands and forearms gripping a barbell. The person is wearing a dark, short-sleeved shirt. The background is bright and out of focus, showing vertical bars or window frames. The text is overlaid on the right side of the image.

Exercício 11.1b)

Comunicação em Rede com Sockets: EchoClient

Exercício 11.1 (b) – EchoClient



Objetivo

Criar um cliente interativo que:

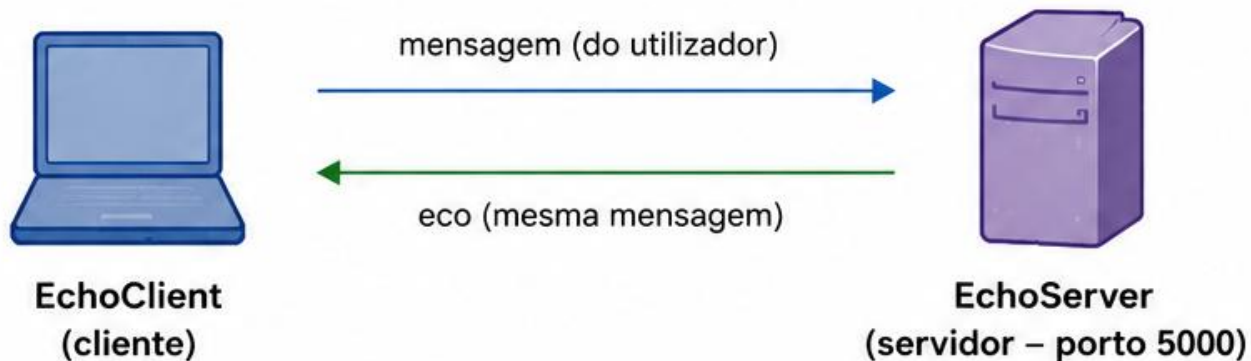
- liga ao EchoServer;
- envia mensagens escritas pelo utilizador;
- recebe o eco do servidor;
- mostra o eco recebido no ecrã.



Descrição

O EchoClient estabelece ligação com o EchoServer e permite ao utilizador enviar mensagens de texto. Para cada mensagem enviada, o servidor devolve exatamente a mesma mensagem (eco), que é mostrada no ecrã.

Funcionamento



Passos do EchoClient

1. Liga ao servidor (Socket).
2. Lê texto do teclado (utilizador).
3. Envia a mensagem ao servidor (println).
4. Recebe o eco do servidor (readLine).
5. Mostra o eco no ecrã.
6. Repete até o utilizador escrever “sair”.
7. Fecha a ligação.

Exemplo de execução

```
> Olá servidor!  
Eco do servidor: Olá servidor!  
> Como estás?  
Eco do servidor: Como estás?  
> sair  
Ligação terminada.
```

Legenda

```
> ...      texto escrito pelo utilizador  
Eco do servidor: ... resposta (eco)
```

Classes principais utilizadas

- Socket – estabelece a ligação ao servidor
- BufferedReader – lê as respostas do servidor
- PrintWriter – envia mensagens ao servidor
- Scanner – lê o texto do teclado (utilizador)

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

1. Package
2. Import
 - java.io.*, para leitura e escrita de dados
 - java.net.*, para comunicação em rede
 - java.util.Scanner, para ler o que o utilizador escreve no teclado.
3. Classe principal
4. Método main
5. Endereço e porto do servidor
6. Ligação ao servidor, tenta ligar o cliente ao servidor
 - Se o servidor estiver ligado, a conexão é criada.
 - Se o servidor não estiver ligado, aparece "Connection refused"
7. Ler mensagens vindas do servidor
8. Para o cliente enviar mensagens para o servidor

```
package aula11;
```

```
import java.io.*;
```

```
import java.net.*;
```

```
import java.util.Scanner;
```

```
public class EchoClient {
```

```
    public static void main(String[] args) {
```

```
        String host = "localhost";
```

```
        int port = 5000;
```

```
        try (Socket socket = new Socket(host, port);
```

```
            BufferedReader in = new BufferedReader(
                new
                InputStreamReader(socket.getInputStream()));
```

```
                PrintWriter out = new
                PrintWriter(socket.getOutputStream(), true);
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

9. Ler do teclado

10. try-with-resources (ponto 6), este try cria vários recursos:

- socket
- leitor
- escritor
- Scanner

11. Mensagens iniciais

12. Variável da mensagem, vai guardar cada mensagem escrita pelo utilizador.

13. Ciclo principal, o cliente continua a:

- pedir uma mensagem
- enviar a mensagem ao servidor
- receber a resposta
- mostrar a resposta

```
Scanner sc = new Scanner(System.in)) {  
  
    System.out.println("Ligado ao  
servidor.");  
    System.out.println("Escreve mensagens.  
Escreve 'sair' para terminar.");  
  
    String msg;  
  
    while (true) {  
        System.out.print("> ");  
        msg = sc.nextLine();  
        out.println(msg);  
        String resposta = in.readLine();  
        System.out.println("Eco do  
servidor: " + resposta);  
        if  
        ("sair".equalsIgnoreCase(msg.trim())) {  
            break;  
        }  
    }  
} catch (IOException e) {  
    System.err.println("Erro no cliente: "  
+ e.getMessage());  
}  
}
```

Exercício 11.1

Comunicação em Rede com Sockets

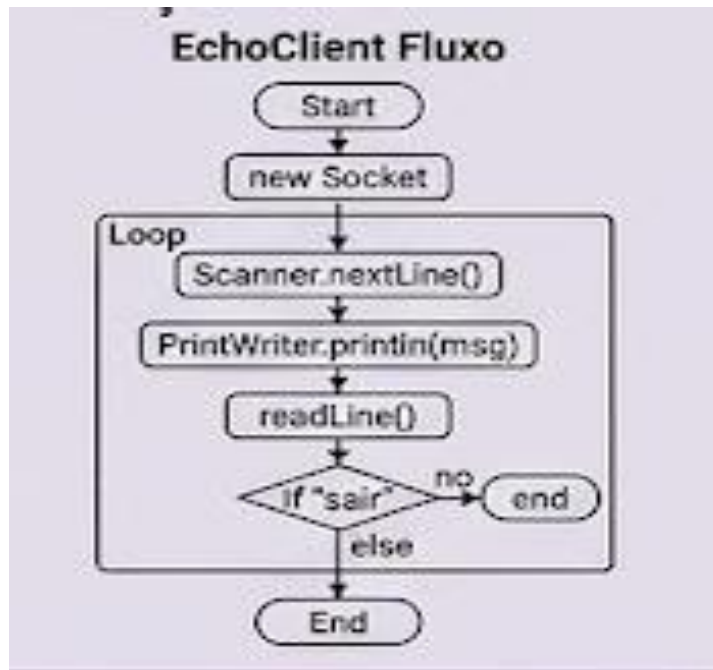
Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

EchoClient.java

Ligado ao servidor.
Escreve mensagens. Escreve 'sair' para terminar.
>



```
package aula11;
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class EchoClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;

        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(
                new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in)) {

            System.out.println("Ligado ao servidor.");
            System.out.println("Escreve mensagens. Escreve 'sair'
para terminar.");
            String msg;
            while (true) {
                System.out.print("> ");
                msg = sc.nextLine();
                out.println(msg);
                String resposta = in.readLine();
                System.out.println("Eco do servidor: " +
resposta);

                if ("sair".equalsIgnoreCase(msg.trim())) {
                    break;
                }
            }
        } catch (IOException e) {
            System.err.println("Erro no cliente: " +
e.getMessage());
        }
    }
}
```

Comunicação Java Cliente-Servidor

Classes Principais

Classes e Suas Funções



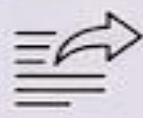
Socket

Estabelece a ligação ao servidor



BufferedReader

Lê as respostas do servidor



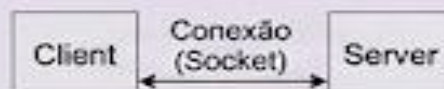
PrintWriter

Envia mensagens ao servidor



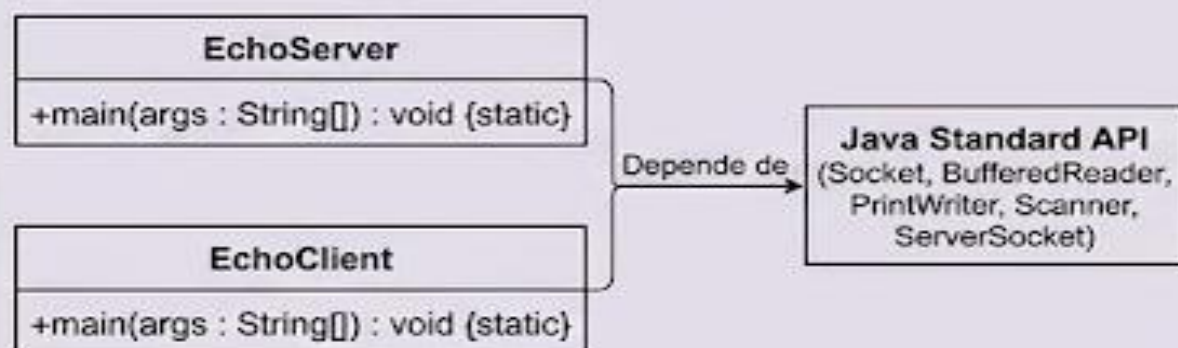
Scanner

Lê o texto do teclado (utilizador)



UML das Classes

UML e Estrutura



Semelhanças e Diferenças

Análise Comparativa

Semelhanças

- Utilização das Mesmas Streams (BufferedReader, PrintWriter)
- Gestão de Exceções (IOException)
- Try-with-Resources

Diferenças

Ponto de Inicialização

ServerSocket

Socket

Papel na Conexão

Passivo (accept)

Activo (new Socket)

Fonte de Input

Rede apenas

Teclado e Rede

Causa de Encerramento

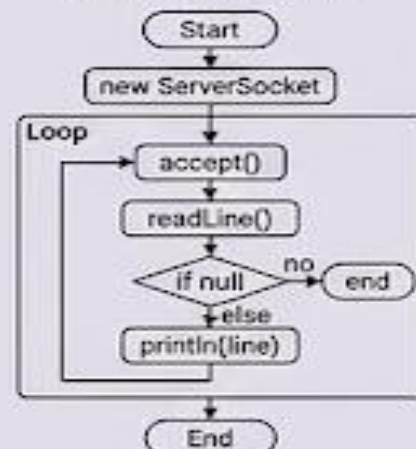
Desconexão do Cliente

Comando "sair"

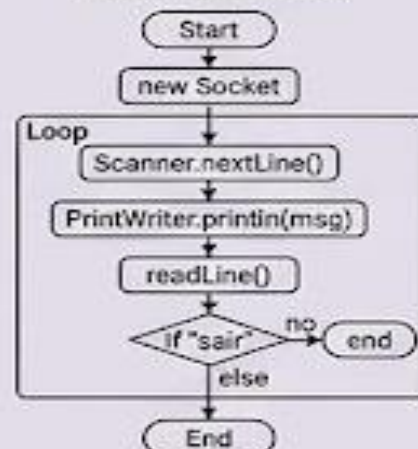
Fluxogramas

Fluxo de Execução

EchoServer Fluxo



EchoClient Fluxo



EchoServer.java

```

package aula11;
import java.io.*;
import java.net.*;
public class EchoServer {
    public static void main(String[] args) {
        int port = 5000;
        System.out.println("A iniciar o Echo Server
no porto " + port + "..."); //feedback visual ao
utilizador

        try (ServerSocket serverSocket = new
ServerSocket(port)) {
            System.out.println("Servidor em escuta.
Aguardando conexao de um cliente...");

            try (Socket clientSocket =
serverSocket.accept();
                BufferedReader in = new BufferedReader(
                    new
InputStreamReader(clientSocket.getInputStream()));
                PrintWriter out = new
PrintWriter(clientSocket.getOutputStream(), true)) {
                System.out.println("Cliente ligado.");
                String line;
                while ((line = in.readLine()) != null) {
                    out.println(line);
                }
            } catch (IOException e) {
                System.err.println("Erro: " +
e.getMessage());
            }
        }
    }
}

```

EchoClient.java

```

package aula11;
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class EchoClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;

        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(
                new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);

                Scanner sc = new Scanner(System.in)) {
            System.out.println("Ligado ao servidor.");
            System.out.println("Escreve mensagens. Escreve
'sair' para terminar.");
            String msg;
            while (true) {
                System.out.print("> ");
                msg = sc.nextLine();
                out.println(msg);
                String resposta = in.readLine();
                System.out.println("Eco do servidor: " +
resposta);

                if ("sair".equalsIgnoreCase(msg.trim())) {
                    break;
                }
            } catch (IOException e) {
                System.err.println("Erro no cliente: " +
e.getMessage());
            }
        }
    }
}

```

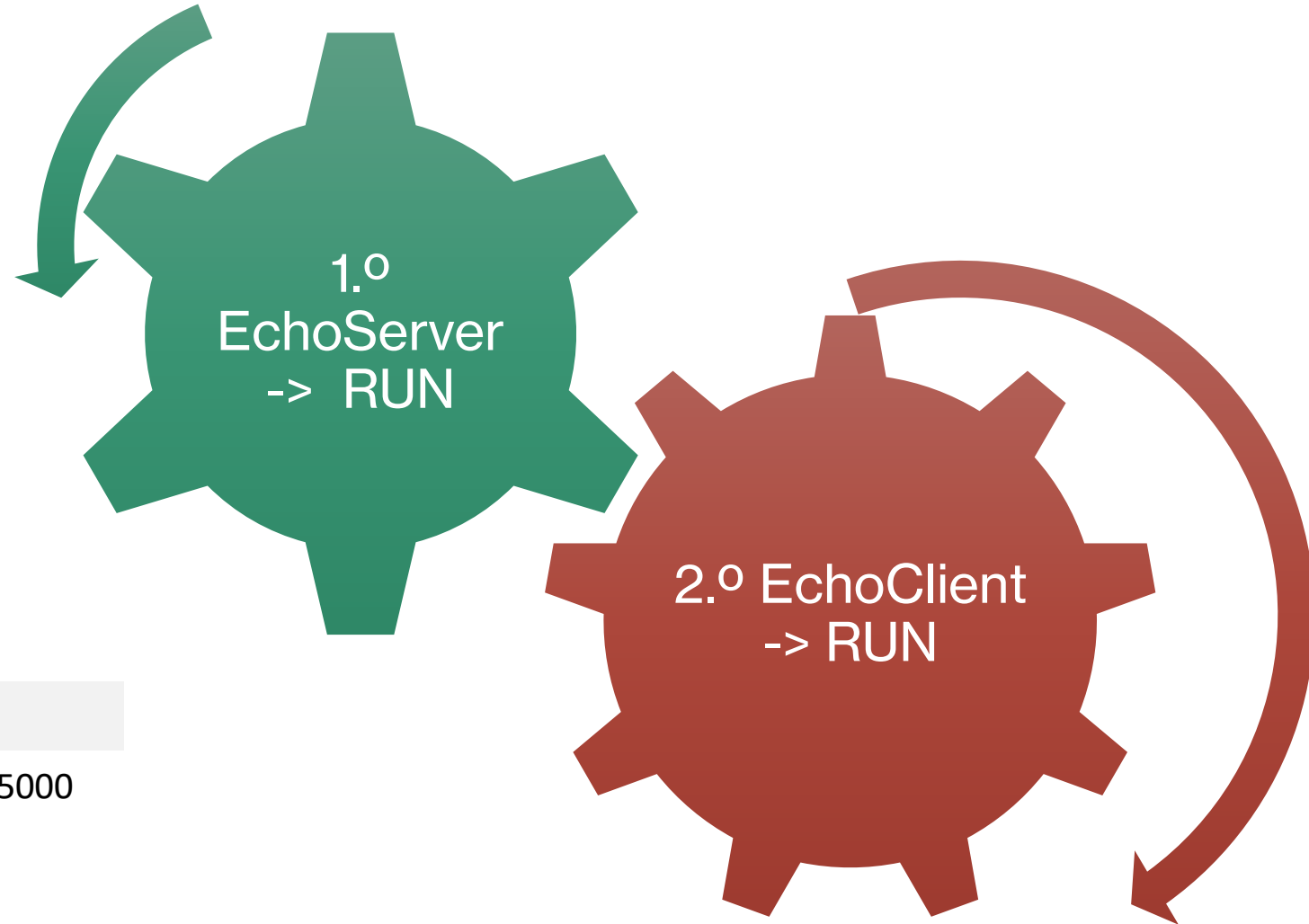
Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



Programa	Função
EchoServer	Fica à espera no porto 5000
EchoClient	Liga-se ao servidor



Exercício 11.1c)

Comunicação em Rede com Sockets: RTTClient

Exercício 11.1 (c) – RTTClient



Objetivo

Criar um cliente que estima o Round-Trip Time (RTT):

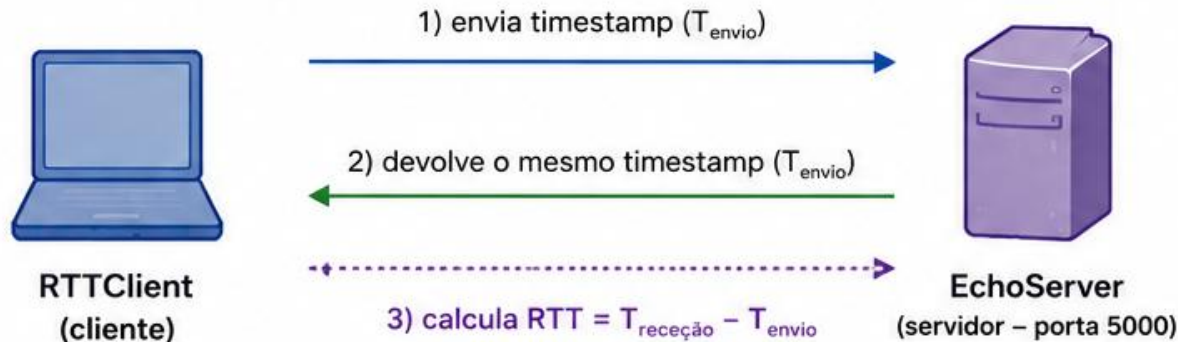
- envia o timestamp atual para o servidor;
- recebe o eco do servidor (mesmo timestamp);
- calcula o tempo de ida e volta (RTT) da mensagem.



O que é RTT?

RTT (Round-Trip Time) é o tempo que uma mensagem leva para ir do cliente ao servidor e voltar ao cliente. É uma medida de latência da comunicação.

Funcionamento



Exemplo de execução (saída típica)

Ligado ao servidor.

RTT = 0,721 ms
RTT = 0,666 ms
RTT = 0,820 ms
RTT = 0,969 ms
RTT = 1,186 ms
RTT = 1,346 ms
RTT = 1,082 ms
RTT = 0,539 ms
RTT = 1,181 ms

Legenda

- mensagem do cliente para o servidor
- resposta do servidor para o cliente
- cálculo do RTT no cliente

Passos do RTTClient (para cada medição)

1. `tempoEnvio = System.nanoTime()` // instante antes do envio
2. Envia tempoEnvio ao servidor (println)
3. Recebe o eco do servidor (readLine)
4. `tempoRececao = System.nanoTime()` // instante após a receção
5. `RTT = tempoRececao - tempoEnvio` // diferença em nanossegundos
6. Converte para milissegundos: `RTT(ms) = RTT / 1_000_000.0`
7. Mostra o RTT no ecrã
8. Repete 10 vezes (com pausa de 1s entre medições)
9. No fim, envia "sair" e fecha a ligação

Fórmula

$$RTT = T_{receção} - T_{envio}$$

onde:

- T_{envio} = instante (nanoTime) antes de enviar a mensagem
- $T_{receção}$ = instante (nanoTime) após receber o eco do servidor



Notas importantes

- Usa-se `System.nanoTime()` para obter instantes com alta precisão (nanossegundos).
- O servidor é o EchoServer do exercício 11.1 (a), que devolve exatamente a mesma mensagem recebida.
- Como o exemplo usa "localhost", o RTT obtido é muito pequeno (sub-milissegundo a poucos milissegundos).

Estrutura de Classes (java.io & java.net)

01

Classes do Pacote java.net (Comunicação em Rede)

- **Socket:** Responsável por estabelecer a ligação TCP activa com o servidor de eco em localhost:5000.

02

Classes do Pacote java.io (Entrada e Saída de Dados)

- **BufferedReader:** Utilizada para ler a resposta textual (o timestamp devolvido) vinda do servidor.
- **InputStreamReader:** Actua como decodificador, convertendo o fluxo de bytes do socket em caracteres.
- **PrintWriter:** Utilizada para enviar o timestamp actual (sob a forma de texto) para o servidor de forma imediata.

03

Classes do Pacote java.lang (Invocadas Implicitamente)

- **System:** Além das saídas de consola (out e err), disponibiliza o método estático **System.nanoTime()**, precisão de nanossegundos.
- **Long:** Utilizada através do método estático Long.parseLong(resposta) para converter a String recebida de volta num tipo primitivo long, permitindo os cálculos matemáticos.
- **Thread:** Utilizada pelo método estático Thread.sleep(1000) para suspender a execução do programa por 1 segundo (1000 milissegundos) entre cada medição de RTT.
- **String e Exception:** Utilizadas para manipulação de texto e para a captura genérica de erros
- os argumentos do método main.

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

RTT= tempo de ida + volta da mensagem

1 medição de RTT.

1. Definir o endereço e porto do servidor
2. Criar a ligação ao servidor
3. Criar o canal de leitura
4. Criar o canal de escrita
5. Mostrar mensagem de ligação
6. **Obter o instante antes do envio**
7. Enviar o timestamp ao servidor
8. Receber o eco do servidor
9. **Obter o instante após a recepção**
10. Converter a resposta de string para número
11. **Calcular o RTT, converter mms e mostrar**

```
package aula11;
import java.io.*;
import java.net.*;

public class RTTClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;
        try {
            Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
                PrintWriter(socket.getOutputStream(), true) {
            System.out.println("Ligado ao servidor.");
            long tempoEnvio = System.nanoTime();
            out.println(tempoEnvio);
            String resposta = in.readLine();
            long tempoRececao = System.nanoTime();
            long tempoRecebido = Long.parseLong(resposta);
            long rttNano = tempoRececao - tempoRecebido;
            double rttMs = rttNano / 1_000_000.0;
            System.out.printf("RTT = %.3f ms%n", rttMs);
            out.println("sair");
            }
        } catch (Exception e) {
            System.err.println("Erro no cliente RTT: " +
                e.getMessage());
        }
    }
}
```

Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)

RTT = tempo de ida + volta da mensagem

10 medições de RTT.

Ligado ao servidor.

RTT = 0,721 ms
RTT = 0,666 ms
RTT = 0,820 ms
RTT = 0,969 ms
RTT = 1,186 ms
RTT = 1,346 ms
RTT = 1,082 ms
RTT = 0,539 ms
RTT = 1,181 ms
RTT = 0,728 ms

o RTT varia ligeiramente, uma medição isolada pode ser enganadora

```
package aula11;
import java.io.*;
import java.net.*;
public class RTTClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;

        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
                PrintWriter(socket.getOutputStream(), true)) {

            System.out.println("Ligado ao servidor.");
            for (int i = 0; i < 10; i++) {
                long tempoEnvio = System.nanoTime();
                out.println(tempoEnvio);

                String resposta = in.readLine();

                long tempoRececao = System.nanoTime();
                long tempoRecebido = Long.parseLong(resposta);
                long rttNano = tempoRececao - tempoRecebido;

                double rttMs = rttNano / 1_000_000.0;
                System.out.printf("RTT = %.3f ms%n", rttMs);
                Thread.sleep(1000);
            }

            out.println("sair");

        } catch (Exception e) {
            System.err.println("Erro no cliente RTT: " +
                e.getMessage());
        }
    }
}
```

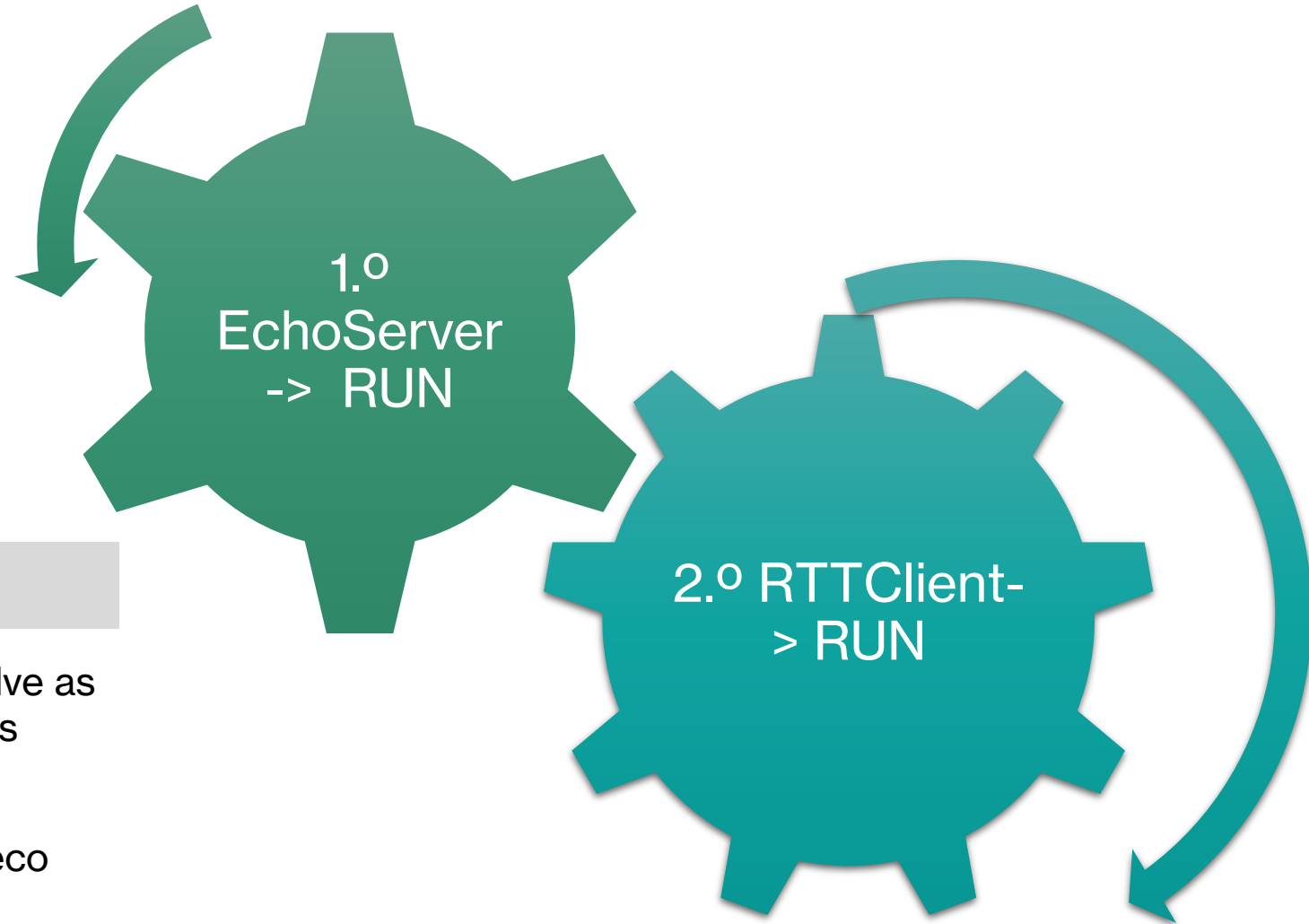
Exercício 11.1

Comunicação em Rede com Sockets

Exercício 11.1 a) Construção de um Echo Server

Exercício 11.1 b): Construção de um Cliente Bidireccional

Exercício 11.1 c): Cálculo de Latência de Rede via Round-Trip-Time (RTT)



Programa	Função
EchoServer	Fica à escuta e devolve as mensagens recebidas
EchoClient	Envia mensagens ao servidor e mostra o eco
RTTClient	Mede o tempo de ida e volta da mensagem (RTT)

RTTClient é um cliente. Ele faz o papel do EchoClient.



Exercício 11.2

Empacotar os clientes em ficheiros JAR executáveis



Exercício 11.2 – Empacotar clientes e usar argumentos



Objetivo

Empacote os clientes anteriores em JARs e tire proveito dos argumentos de entrada do método main para parametrizar o endereço e porto do servidor a ser contactado. Coloque em execução o servidor e teste os seus JARs de clientes.



Ideia principal

O cliente deve receber, pela linha de comandos, o endereço (host) e o porto do servidor. Dessa forma, o mesmo JAR pode ser usado para ligar a diferentes servidores.

Argumentos do main

O cliente recebe os argumentos da linha de comandos:

- args[0] – endereço do servidor (host)
- args[1] – porto do servidor (port)

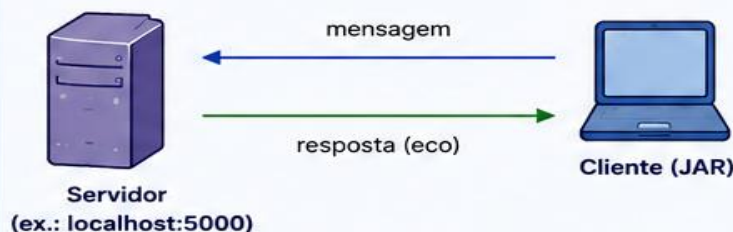
```
public class Cliente {
    public static void main(String[] args) {
        String host = "localhost"; // valor por defeito
        int port = 5000;           // valor por defeito

        if (args.length >= 1) {
            host = args[0];
        }
        if (args.length >= 2) {
            port = Integer.parseInt(args[1]);
        }

        System.out.println("Ligar a " + host + ":" + port);

        // ... resto do código do cliente
    }
}
```

Como funciona



Exemplo de execução

```
$ java -jar cliente.jar
Ligar a localhost:5000
Mensagem: Olá servidor!
Eco do servidor: Olá servidor!
```

```
$ java -jar cliente.jar 192.168.1.10 6000
Ligar a 192.168.1.10:6000
Mensagem: Teste
Eco do servidor: Teste
```

Passos para o aluno



- 1 Altere o cliente para ler os argumentos (host e port) do método main.
- 2 Compile o projeto.
- 3 Empacote o cliente num JAR executável.
- 4 Coloque o servidor em execução.
- 5 Execute o cliente usando o JAR, passando os argumentos desejados.
- 6 Teste com diferentes endereços e portas.

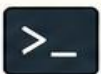
Empacotar cliente em JAR

Exemplo de criação de JAR executável:

```
jar cfe cliente.jar aula11.Cliente bin/aula11/*.class
```

Onde:

- cliente.jar – nome do JAR a criar
- aula11.Cliente – classe com método main
- bin/aula11/*.class – classes compiladas do pacote aula11



Compilar

```
$ javac -d bin src/aula11/*.java
# classes compiladas ficam em bin/aula11/
```



Executar o servidor

```
$ java -cp bin aula11.EchoServer
# ou o nome da classe do servidor usada
```



Executar o cliente (JAR)

```
$ java -jar cliente.jar # usa valores por defeito (localhost:5000)
$ java -jar cliente.jar localhost 5000 # especifica host e porta
$ java -jar cliente.jar 192.168.1.10 6000
```



Resumo

O cliente é empacotado num JAR executável e recebe os argumentos (endereço e porto) pela linha de comandos, permitindo ligar a diferentes servidores sem recompilar o programa.



Exercício 11.2

Empacotar os clientes em ficheiros JAR executáveis

Empacote os clientes anteriores em JARs e tire proveito dos argumentos de entrada do método main para parametrizar o endereço e porto do servidor a ser contactado.

Coloque em execução o servidor e teste os seus JARs de clientes.

Exemplo:

```
jar cfe cliente.jar aula11.ClasseComMain bin/aula11/*.class
```

Um ficheiro **JAR (Java ARchive)** é um ficheiro que agrupa:

- classes compiladas (.class)
 - bibliotecas
 - recursos do programa
- num único ficheiro executável.



.java → código fonte.
class → código compilado.
jar → programa empacotado

Objectivo?

Empacotar os clientes em ficheiros JAR executáveis e permitir escolher, pela linha de comandos:

- endereço do servidor
- porto do servidor



Sem JAR

Projeto/
├── EchoClient.class
├── EchoServer.class
├── RTTClient.class
└── ...

-> muitos ficheiros separados

Com JAR

cliente.jar

-> tudo empacotado num único ficheiro

pode ser executado por: `java -jar cliente.jar`

Exercício 11.2

Empacotar os clientes em ficheiros JAR executáveis

Empacote os clientes anteriores em JARs **e tire proveito dos argumentos de entrada do método main para parametrizar o endereço e porto do servidor a ser contactado.**

Coloque em execução o servidor e teste os seus JARs de clientes.

Exemplo:

```
jar cfe cliente.jar aula11.ClasseComMain bin/aula11/*.class
```

Antes (cliente anterior)

O endereço e porto estavam fixos no código:

```
ServerSocket serverSocket =  
new ServerSocket(port)
```

O cliente ligava sempre ao mesmo servidor; obrigava a alterar o código para mudar o porto/endereço.

```
package aula11;  
import java.io.*;  
import java.net.*;  
public class EchoClient2 {
```

o main não mudou;
mudou foi o comportamento do programa.

```
    public static void main(String[] args) {
```

```
        String host = "localhost";
```

```
        int port = 5000;
```

```
        if (args.length >= 1) {
```

```
            host = args[0];
```

```
        }
```

```
        if (args.length >= 2) {
```

```
            port = Integer.parseInt(args[1]);
```

```
        }
```

```
        try {
```

```
            Socket socket = new Socket(host, port);
```

```
            BufferedReader in = new BufferedReader(  
                new
```

```
                InputStreamReader(socket.getInputStream()));
```

```
            PrintWriter out = new
```

```
            PrintWriter(socket.getOutputStream(), true)  
            ) {
```

```
                out.println("Teste");
```

```
                String resposta = in.readLine();
```

```
                System.out.println("Resposta do servidor: " +  
resposta);
```

```
            } catch (IOException e) {
```

```
                System.out.println("Erro: " + e.getMessage());
```

```
            }  
        }  
    }
```

Exercício 11.2

Empacotar os clientes em ficheiros JAR executáveis

Empacote os clientes anteriores em JARs e tire proveito dos argumentos de entrada do método main para parametrizar o endereço e porto do servidor a ser contactado.

Coloque em execução o servidor e teste os seus JARs de clientes.

Exemplo:

```
jar cfe cliente.jar aula11.ClasseComMain bin/aula11/*.class
```

PASSO 1 → Criar o JAR

PASSO 2 → Executar servidor

PASSO 3 → Executar cliente

Servidor e cliente são programas diferentes.
Por isso são executados em terminais separados.
O servidor fica continuamente à escuta.

1. Criar o JAR executável

```
jar cfe EchoClient2.jar aula11.EchoClient2 -C  
bin aula11
```

```
PS C:\P00\Exercicios_RMA> jar cfe EchoClient2.jar aula11.EchoClient2 -C bin aula11
```

TERMINAL 1

2. Colocar em execução o servidor

```
java -cp bin aula11.EchoServer
```

```
PS C:\P00\Exercicios_RMA> java -cp bin aula11.EchoServer  
A iniciar o Echo Server no porto 5000...  
Servidor em escuta. Aguardando conexao de um cliente...  
█
```

TERMINAL 2

3. Empacotar clientes em JARs executáveis

```
java -jar cliente.jar localhost 5000
```

```
PS C:\P00\Exercicios_RMA\src> java -jar cliente.jar localhost 5000  
Error: Unable to access jarfile cliente.jar  
PS C:\P00\Exercicios_RMA\src> cd ..  
PS C:\P00\Exercicios_RMA> java -jar cliente.jar localhost 5000  
Resposta do servidor: Teste  
PS C:\P00\Exercicios_RMA> █
```

Exercício 11.2

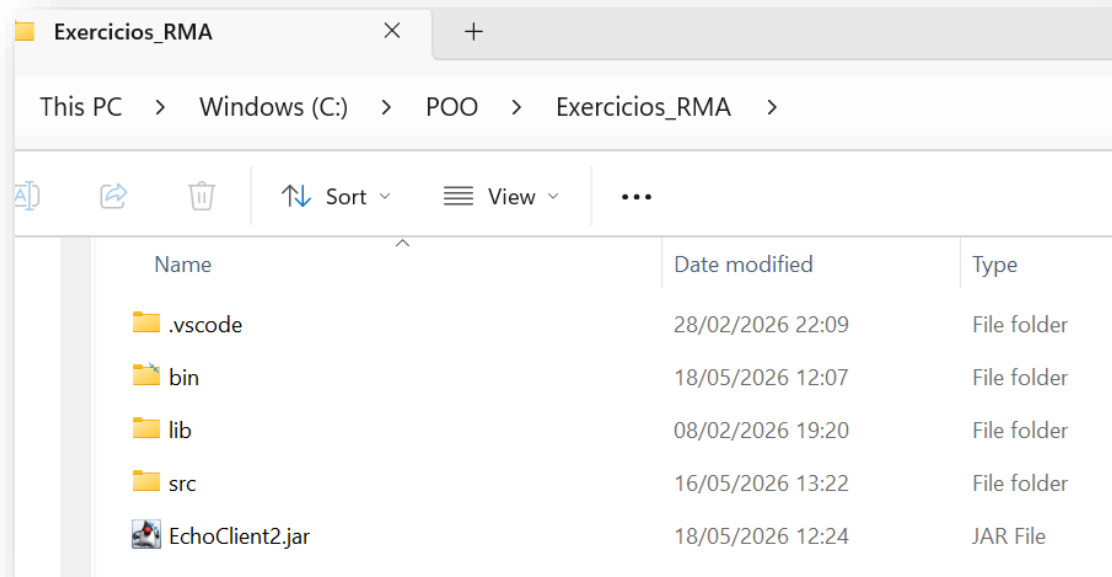
Empacotar os clientes em ficheiros JAR executáveis

Empacote os clientes anteriores em JARs e tire proveito dos argumentos de entrada do método main para parametrizar o endereço e porto do servidor a ser contactado.

Coloque em execução o servidor e teste os seus JARs de clientes.

Exemplo:

```
jar cfe cliente.jar aula11.ClasseComMain bin/aula11/*.class
```



O JAR serve para empacotar e distribuir um programa Java executável.

o exercício é MINIMALISTA.

Onde isto faz MESMO sentido?

Exemplo:

um servidor numa máquina;

20 alunos noutras máquinas;

- cada um só recebe: EchoClient2.jar
- executa: java -jar EchoClient2.jar 192.168.1.50 5000

**não precisam do código;
nem do IDE;
nem do projecto Java.
Só do JAR.**

Exercício 11.2

Empacotar clientes em JARs executáveis

Empacote os clientes anteriores em JARs e tire proveito dos argumentos de entrada do método `main` para parametrizar o endereço e o porto do servidor a ser contactado.

IDEIA PRINCIPAL

Cliente e servidor são programas diferentes.
Por isso são executados em terminais separados.
O servidor fica continuamente à escuta.

PASSOS

- 1 Criar o JAR do cliente
- 2 Executar o servidor
- 3 Executar o cliente empacotado

PARA QUE SERVE?

- Distribuir o cliente como um ficheiro executável (.jar) que pode ser executado sem o IDE.
- Permitir configurar o endereço e o porto do servidor na linha de comandos.
- O mesmo cliente pode ligar a diferentes servidores.

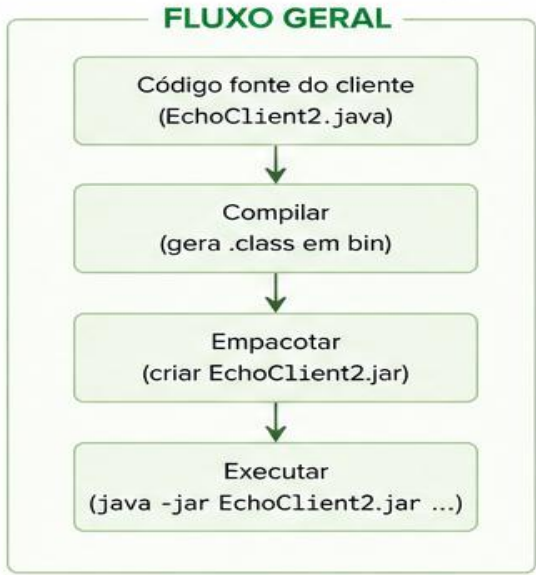
1. Criar o JAR executável

Empacote o cliente (ex.: `EchoClient2`) num JAR.
Este passo só é necessário quando o código é alterado (e precisa de recriar o JAR).

```
jar cfe EchoClient2.jar aula11.EchoClient2 -C bin aula11
```

Exemplo de ficheiros no projeto

- .vscode
- bin
- lib
- src
-  EchoClient2.jar



2. Executar o servidor

Terminal 1

Inicie o servidor. Ele fica à escuta no porto 5000.

```
java -cp bin aula11.EchoServer
```

```
PS C:\P00\Exercicios_RMA> java -cp bin aula11.EchoServer
A iniciar o Echo Server no porto 5000...
Servidor em escuta. Aguardando conexao de um cliente...
|
```

3. Executar o cliente empacotado

Terminal 2

Execute o cliente empacotado em JAR, indicando o endereço e o porto do servidor.

```
java -jar EchoClient2.jar localhost 5000
```

```
PS C:\P00\Exercicios_RMA> java -jar EchoClient2.jar localhost 5000
Resposta do servidor: Teste
PS C:\P00\Exercicios_RMA>
```

NOTAS IMPORTANTES

- O JAR é apenas do CLIENTE (neste exercício). O servidor continua a ser executado separadamente.
- Cada cliente executável deve ter o seu próprio JAR (ex.: `EchoClient.jar`, `EchoClient2.jar`).
- O JAR contém todas as classes necessárias e sabe qual é a classe principal (`aula11.EchoClient2`).

RESUMO

Criamos um JAR executável do cliente para poder executá-lo fora do IDE e parametrizá-lo pela linha de comandos. Executamos o servidor num terminal (à escuta) e o cliente empacotado noutro terminal (que se liga ao servidor).